

DOSSIER PROJET



Nom de naissance : Chemin

Prénom : Guillaume

Adresse : 3 résidence des Clos Mullons 28300 Lèves

Titre professionnel visé : Concepteur
Développeur d'applications

N° Fiche AT	Activités types	N° Fiche CP	Compétences professionnelles
1	Concevoir et développer des composants d'interface utilisateur en intégrant les recommandations de sécurité	1	Maquetter une application
		2	Développer une interface utilisateur de type desktop
		3	Développer des composants d'accès aux données
		4	Développer la partie front-end d'une interface utilisateur web
		5	Développer la partie back-end d'une interface utilisateur web
2	Concevoir et développer la persistance des données en intégrant les recommandations de sécurité	6	Concevoir une base de données
		7	Mettre en place une base de données
		8	Développer des composants dans le langage d'une base de données
3	Concevoir et développer une application multicouche répartie en intégrant les recommandations de sécurité	9	Collaborer à la gestion d'un projet informatique et à l'organisation de l'environnement de développement
		10	Concevoir une application
		11	Développer des composants métier
		12	Construire une application organisée en couches
		13	Développer une application mobile
		14	Préparer et exécuter les plans de tests d'une application
		15	Préparer et exécuter le déploiement d'une application

1. Résumé du projet en anglais.....	P4
2.Cahier des charges.....	P6
2.1 Description des charges du projet.....	P6
2.2 Les utilisateurs du projet.....	P7
3.Base de données.....	P12
3.1 Résumé de la gestion des données.....	P12
3.2 Concevoir et développer une base de données.....	P13
3.3 Développer des composants d'accès aux données.....	P17
4.Le front-end.....	P22
4.1 Les différentes pages HTML.....	P22
4.2 Création de la structure HTML de base.....	P22
4.3 Création de l'en-tête (Header).....	P23
4.4 Création de la barre latérale.....	P24
4.5 Création du contenu dynamique.....	P25
4.6 Création du pied de page (Footer).....	P25
4.7 CSS pour styliser la page.....	P27
5.Le back-end.....	P29
5.1 Partie récolte des données.....	P29
5.2 Mise en œuvre.....	P32
5.3 Fonctionnalité : Remplissage de PDF.....	P36
5.4 Fonctionnalité : Scraping avec BeautifulSoup.....	P38
6.Déploiement de l'application.....	P40
6.1 Achat du nom de domaine.....	P41
6.2 Mise en place du protocole HTTPS.....	P41
6.3 Hébergement et DNS.....	P41
6.4 Déploiement des sources et configuration finale.....	P41
7.Après projet.....	P41
7.1 ANSSI.....	P41
7.2 Stack Overflow.....	P42
7.3 GitHub.....	P42
8.Application desktop : Gestionnaire de mots de passe.....	P42
8.1 Description de l'application.....	P43
8.2 Base de données de l'application.....	P45
8.3 Fonction pour s'enregistrer (Log in).....	P46
8.4 Changer un mot de passe : Update a password.....	P47
8.5 Technologies utilisées.....	P48
9.Annexes.....	P49

1. Résumé du projet en anglais :

In this project, we aimed to automate the process of filling out CERFA documents, which are notoriously time-consuming and error-prone, often requiring up to 10 hours per document. Our solution dramatically reduces the time needed for completion. The key innovation lies in an automated system that fills out these forms. This is achieved through a series of questionnaires designed to gather and scrape relevant information. The entire process is streamlined and managed using a Flask application, providing a user-friendly and efficient interface. Additionally, at the client's request, we integrated no-code/low-code tools such as Make and Tally. These tools enhance the flexibility and accessibility of our solution, catering to users with varying levels of technical expertise. The end result is a highly efficient, automated system that significantly cuts down the time and effort required to fill out CERFA documents, while also minimizing errors.

In addition to the core features of our project, we employed a range of technologies to enhance the user experience and efficiency of the system. For the front-end development, we utilized HTML Bootstrap, ensuring a responsive and intuitive interface. This choice allowed us to create a visually appealing and easy-to-navigate layout. The back-end of the application was powered by Flask, a lightweight and versatile web framework perfect for our needs. For styling, CSS was employed to give the application a polished and professional look, aligning with the user-friendly design philosophy. Finally, the database, a crucial component of our system, was developed using MySQL. This robust and reliable database management system was chosen for its scalability and security features, ensuring that our application could handle large volumes of data with ease and maintain the integrity of sensitive information. Together, these technologies formed a cohesive and efficient system, streamlining the CERFA document automation process.

2. Cahier des charges

a. Description des charges du projets

Le projet de la Nao School vise à développer une application innovante destinée à automatiser le remplissage des documents CERFA. La demande spécifique du client consiste à créer une application unique, simplifiant considérablement le processus administratif actuel.

L'application fonctionnera de manière intuitive : en cliquant sur un bouton, les utilisateurs déclencheront l'envoi de questionnaires spécifiques. Ces questionnaires, une fois remplis, alimentent automatiquement une base de données centrale. Lorsque toutes les informations nécessaires seront collectées, l'application se chargera de remplir les formulaires CERFA de manière automatisée, éliminant ainsi le besoin de saisie manuelle fastidieuse.

Les principaux utilisateurs de cette application seront les employés de la Nao School. Ils auront accès à l'interface pour gérer et superviser le processus. Cependant, les autres acteurs impliqués, tels que les écoles et les élèves, recevront les questionnaires directement par e-mail, sans nécessiter un accès direct à l'application. Cette approche garantit une

facilité d'utilisation et une accessibilité maximales pour tous les participants, tout en centralisant la gestion et le suivi du processus au sein de l'équipe de la Nao School.

Ce système apportera plusieurs avantages significatifs :

- **Efficacité accrue** : L'automatisation du remplissage des documents CERFA réduira considérablement le temps et les efforts nécessaires à la gestion administrative.
- **Fiabilité améliorée** : En minimisant les interventions manuelles, l'application réduit les risques d'erreurs, garantissant ainsi la précision des informations soumises.
- **Suivi simplifié** : Avec toutes les données centralisées, les employés de la Nao School pourront facilement surveiller et gérer le processus de remplissage des CERFA.

Le client a spécifiquement demandé l'intégration de technologies low-code, telles que Make et Tally, dans le développement de l'application. Cette exigence découle de sa familiarité et de sa maîtrise de ces outils. L'utilisation de ces plateformes low-code permet au client de modifier facilement les questionnaires si les formulaires CERFA évoluent, offrant ainsi une grande flexibilité et adaptabilité. De plus, le choix de Google Cloud Platform (GCP) pour héberger la base de données s'aligne également avec les compétences techniques du client. Cette décision garantit non seulement une intégration harmonieuse avec les outils Make, mais aussi une gestion efficace et sécurisée des données.

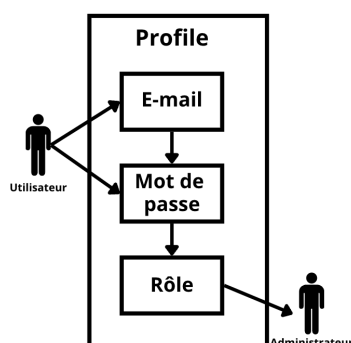
b. Les utilisateurs du projet

Dans le cadre de notre projet d'automatisation des formulaires CERFA pour la Nao School, les utilisateurs se répartissent en plusieurs catégories, chacune avec des niveaux d'accès et de fonctionnalités distincts :

- **Les Administrateurs** : Ils jouent un rôle décisionnel dans l'application, possédant une vue d'ensemble des fonctionnalités sans nécessairement avoir des compétences techniques en codage. Ils ont un accès complet à toutes les fonctionnalités de l'application, leur permettant de superviser et de gérer l'ensemble du processus.
- **Les Acteurs Externes (Écoles et Élèves)** : Ce groupe comprend les utilisateurs qui recevront les questionnaires par e-mail. Ils ne nécessitent pas d'accès direct à l'application dans le début. Leur interaction se limite à fournir les informations demandées via les questionnaires. Cette méthode simplifie leur participation, tout en centralisant la collecte et la gestion des données au sein de la Nao School.

Chaque catégorie d'utilisateurs a été conçue pour répondre aux besoins spécifiques de son rôle dans le processus, garantissant ainsi un fonctionnement efficace et sécurisé de l'application. Il est essentiel de prendre en compte la facilité d'utilisation pour les acteurs externes, qui peuvent ne pas être à l'aise avec les technologies informatiques, et de

s'assurer que les données reçues sont traitées avec prudence pour éviter les erreurs ou les abus.



2. Maquettage et fonctionnalités

2.1 Fonctionnalités :

L'application que nous avons développée pour la Nao School fonctionne de la manière suivante :

Création du Compte : L'utilisateur (l'employé de la Nao School) commence par créer un compte dans l'application. Cette étape nécessite la saisie d'informations de base pour établir son profil et ses droits d'accès.

Authentification : Une fois le compte créé, l'utilisateur doit s'authentifier à chaque accès à l'application. Cette mesure de sécurité garantit que seuls les utilisateurs autorisés peuvent accéder aux données sensibles.

Tableau de Bord : Après l'authentification, l'utilisateur accède au tableau de bord principal de l'application. Ce tableau de bord présente un aperçu des différentes catégories de données :

Données des Écoles : Informations relatives aux établissements partenaires.

Données des Élèves : Détails concernant les élèves inscrits.

Données des Entreprises : Informations sur les entreprises collaboratrices.

Données des Sessions : Aperçu des sessions de formation ou événements organisés.

Plusieurs fonctionnalités sont également prévues par l'application afin de faire gagner du temps et de ce fait de l'argent à l'école Nao School :

- Scraping des informations disponibles sur internet concernant les entreprises afin de leur faire gagner du temps dans le remplissage des contrats CERFA.
- Remplissage automatique d'un PDF à partir de données.
- Téléchargement de ce PDF.

2.2 Maquette de l'application

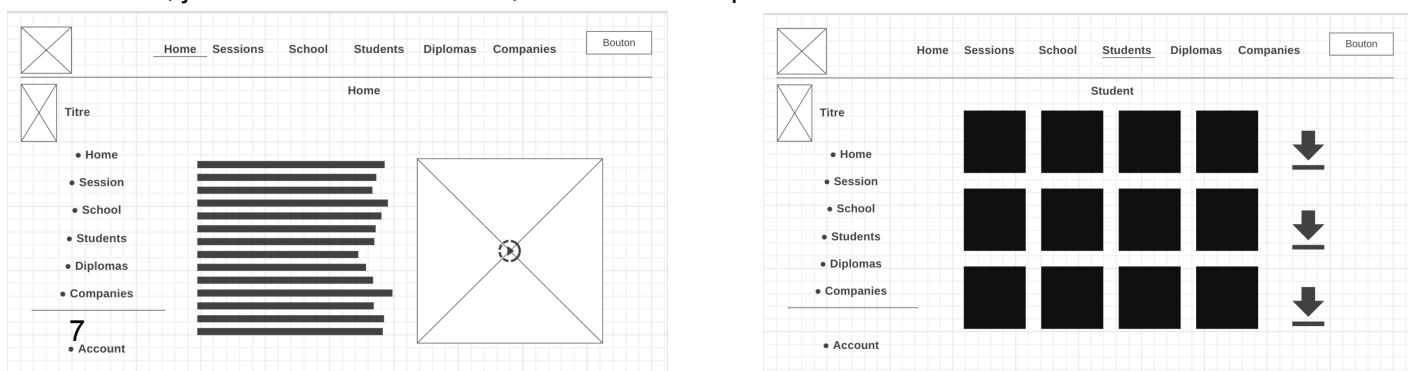
J'ai réalisé des maquettes pour présenter un projet de remplissage automatique de formulaires CERFA. Le processus a été structuré en trois étapes distinctes, avec des retours du client à chaque phase. Le projet comprend trois pages principales, chacune jouant un rôle clé dans l'interface utilisateur :

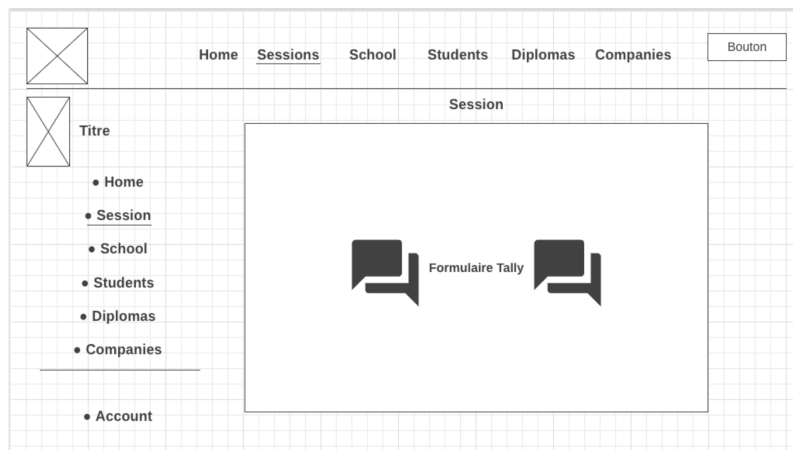
1. **Le menu** : Il s'agit de la page d'accueil permettant de naviguer entre les différentes fonctionnalités de l'application.
2. **Affichage des données** : Cette page permet de visualiser les données extraites ou saisies. Elle inclut également une option pour télécharger les fichiers PDF générés automatiquement.
3. **Affichage du formulaire** : Ici, l'utilisateur peut visualiser et interagir avec le formulaire CERFA, avec la possibilité de remplir ou modifier les champs avant de finaliser le processus.

J'ai d'abord conçu un **zoning**, permettant de définir l'organisation générale des éléments sur la page.

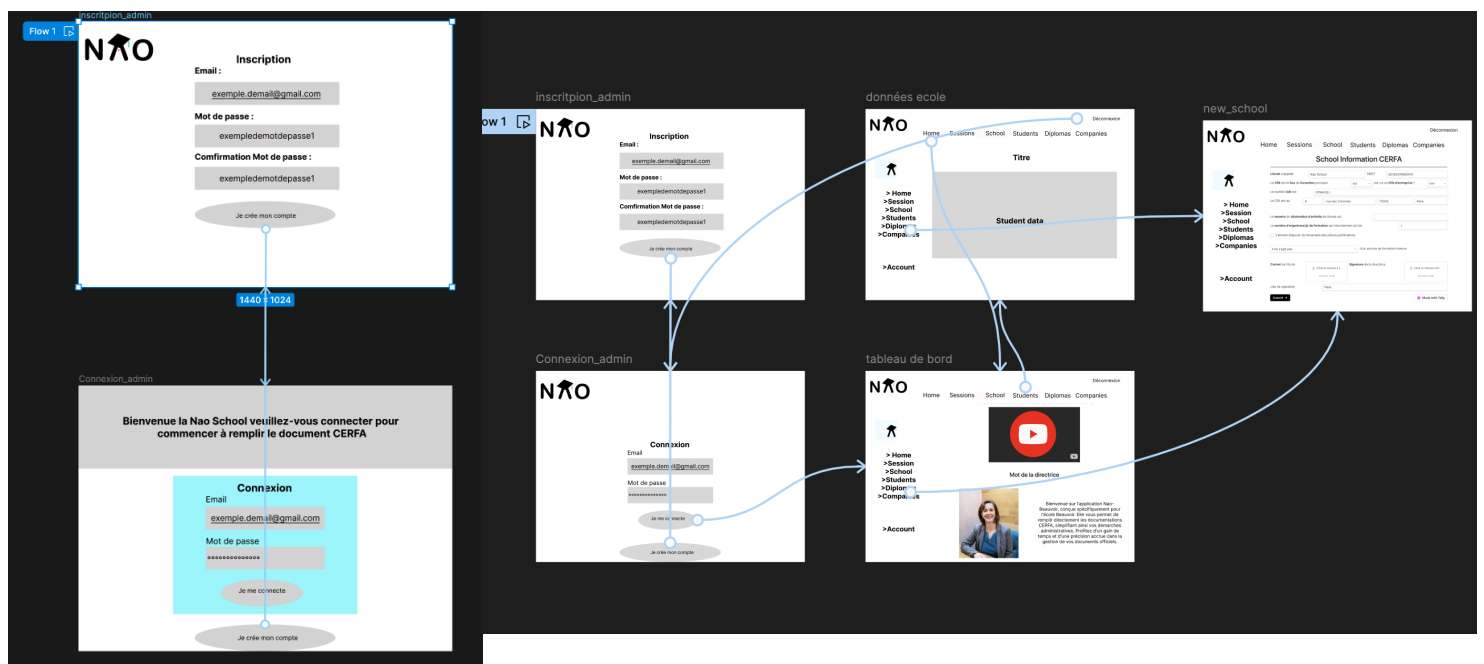


Ensuite, j'ai élaboré un **wireframe**, offrant une vue plus détaillée.



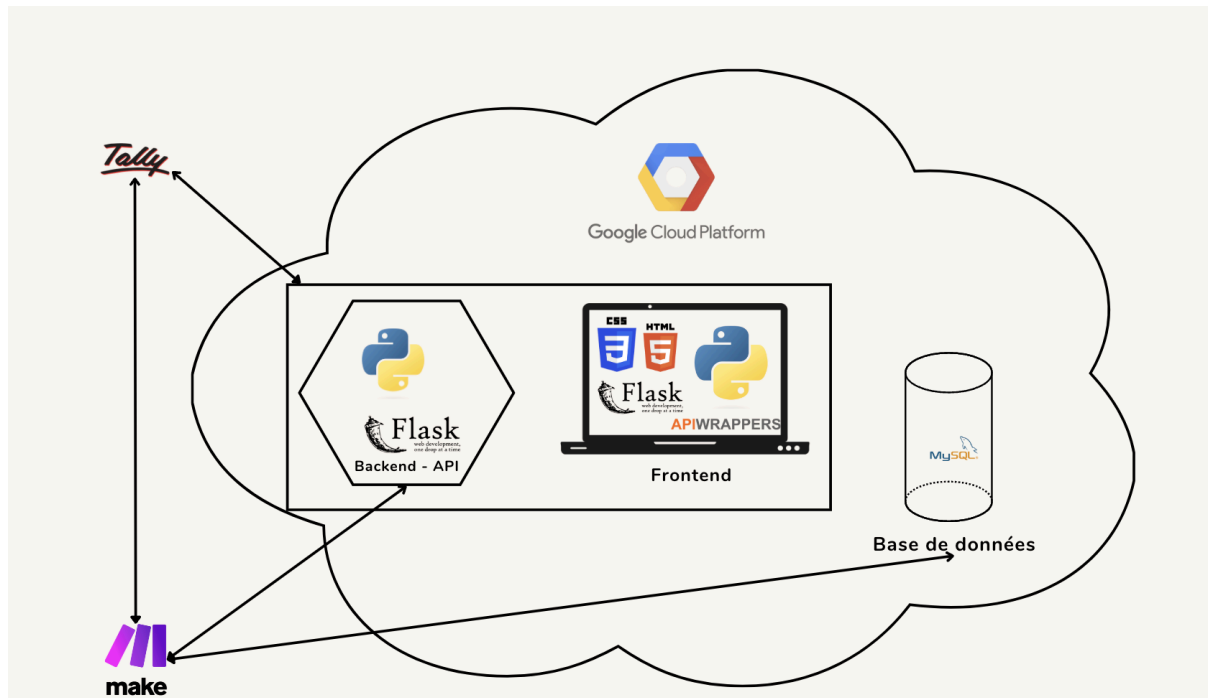


Enfin, j'ai finalisé le projet avec un **mockup**, intégrant le design visuel complet et prêt à être validé par le client. Ces étapes ont permis d'assurer une conception progressive et collaborative.



2.3 Architecture + modèle MVC

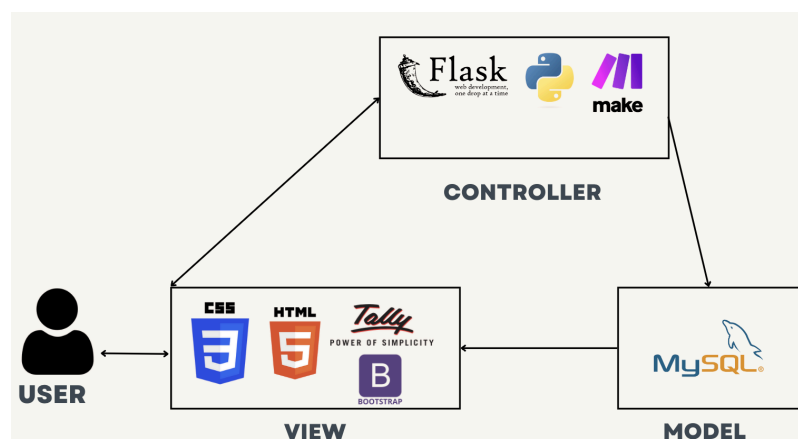
Voici un exemple d'architecture de notre projet



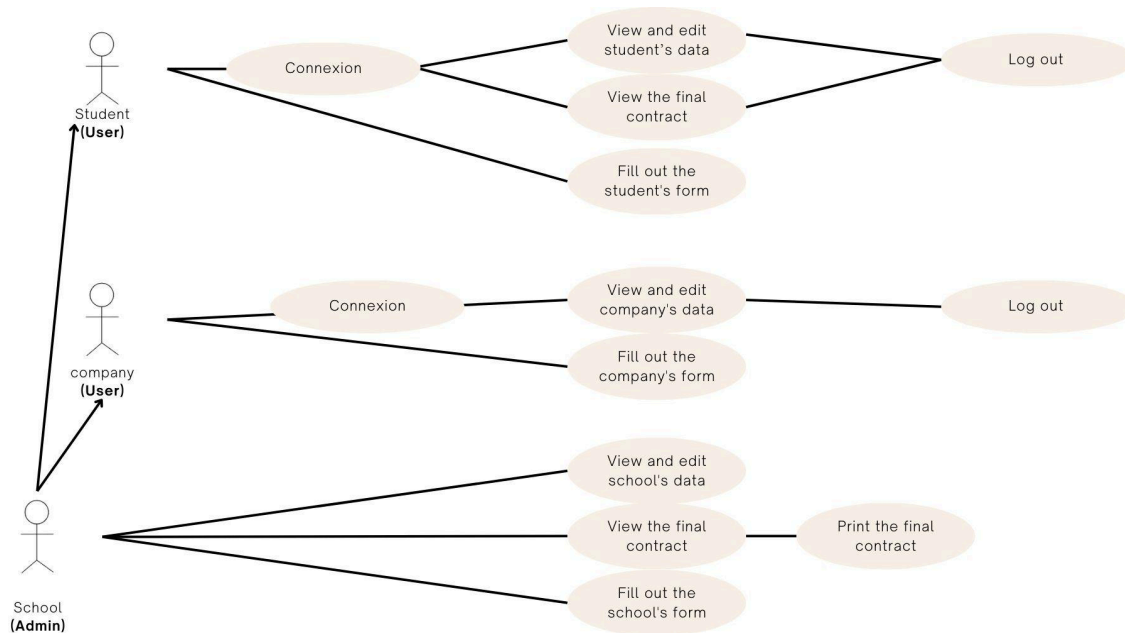
Je me suis ensuite penché sur la conception technique de l'application en suivant un modèle MVC (Modèle-Vue-Contrôleur) pour assurer une séparation des responsabilités entre les différentes couches. Le schéma MVC sera intégré à cette étape pour illustrer clairement l'architecture.

Le modèle MVC m'a permis de structurer l'application en trois couches principales :

- **Modèle** : Gestion de la logique métier et des données (base de données MySQL, Couche d'Accès aux Données)
- **Vue** : Interaction avec les utilisateurs via des templates HTML et CSS. (Couche de présentation)
- **Contrôleur** : Traitement des requêtes et envoi des réponses aux vues, en interaction avec les modèles. (Couche Logique Métier)



Et enfin voici les Uses Cases :



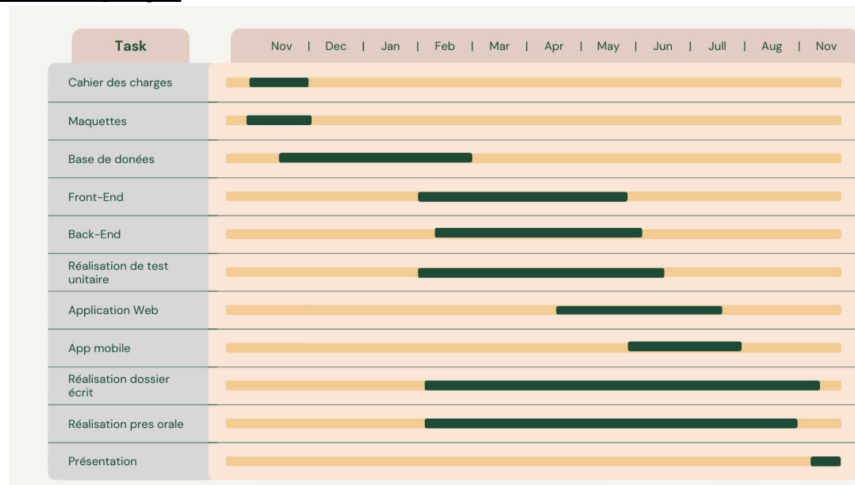
à noter :

- Au départ, nous ne donnerons l'accès qu'aux administrateurs. Les étudiants ainsi que les entreprises rempliront les données via des formulaires Tally qui leur seront envoyés directement sur leurs boîtes e-mail.

Voici la charte graphique :



2.4 : Calendrier du projet



2.5 gestion du projet

Pour organiser et structurer le développement de l'application, nous avons utilisé la méthode **Scrum**, qui fait partie des méthodologies Agiles. Cette méthode nous a permis de diviser le projet en **sprints** (périodes de développement courtes et intensives) afin de livrer des fonctionnalités à chaque itération.

Tout au long du projet, j'ai joué le rôle de **Scrum Master** et de **développeur**, ce qui m'a permis de :

- Animer des réunions quotidiennes (daily stand-ups) pour faire le point sur les avancées du projet.
- Prioriser les tâches
- Échanger avec le client sur l'application et l'adapter en fonction de ses retours

Dans le cadre de ce projet, nous avons utilisé **GitHub** pour la gestion du code source et la collaboration. Disposer d'un **compte GitHub** a grandement facilité le travail en équipe, en nous permettant de versionner le code, de suivre les modifications, et d'intégrer les contributions de chaque membre de manière structurée et organisée.

```
git clone https://github.com/nom-utilisateur/nom-du-depot.git
```

Cette commande bash sert par exemple à installer un projet sur son ordinateur

3. Base de donnée

3.1 Résumé de la gestion des données :

Cette partie présente les fonctionnalités principales de notre application. Elle détaille les accès aux données, la génération automatique de contrats, ainsi que le processus d'ajout et de gestion des informations des différentes parties prenantes. L'administration de l'école est la seule à pouvoir consulter et modifier les données, assurant ainsi leur exactitude avant l'envoi des contrats aux parties concernées.

Accès aux Données :

1. Données de l'École et des Sessions :

L'utilisateur peut accéder à cette section pour consulter et gérer les informations relatives à son école. Il a également la possibilité d'ajouter une session directement via l'application. Après toute consultation ou modification, il peut revenir au menu principal.

2. Données des Entreprises :

De manière similaire, l'utilisateur peut consulter les informations des entreprises. Après consultation, il peut retourner au menu principal.

3. Données des Étudiants et Génération des Contrats :

Cette section permet de visualiser les données des étudiants. Une fois que tous les étudiants ont complété leurs questionnaires, l'application génère automatiquement les contrats correspondants, facilitant ainsi le processus administratif tout en garantissant l'exactitude et la conformité des documents produits.

Processus d'Ajout et de Gestion des Étudiants :

Lorsque l'étudiant est ajouté à l'application, des questionnaires sont envoyés progressivement aux différentes parties concernées, notamment les écoles et les entreprises partenaires. Ces questionnaires recueillent des informations essentielles, qui sont directement intégrées dans la base de données. Ce processus automatisé garantit des données à jour et précises, simplifiant ainsi la gestion et le suivi des informations relatives aux étudiants, aux établissements éducatifs et aux entreprises collaboratrices.

Ce système de collecte en temps réel permet une gestion dynamique et réactive des informations au sein de l'application.

Droits et Modifications des Données :

Seule l'administration de l'école a accès à ces données. Elle peut les modifier en cas d'erreurs signalées par elle-même ou par un tiers lors de la génération d'un contrat.

Envoi des Contrats :

Une fois le contrat généré, il suffit de l'envoyer aux différentes parties pour signature.

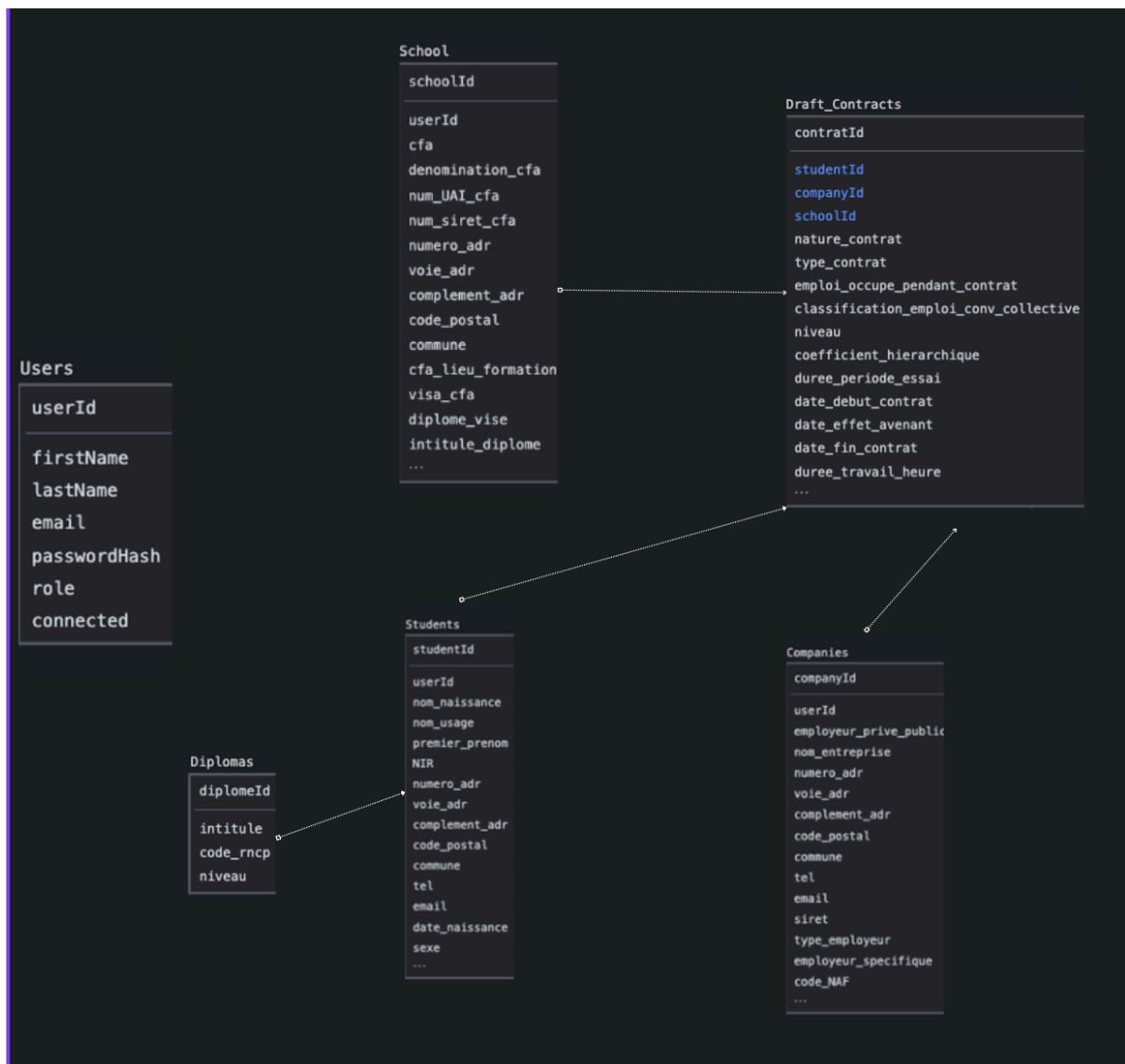
Voici la base de donnée dans son entièreté

Users	Students	School	Companies	Draft_Contracts	Diplomas
userId	studentId	schoolId	companyId	contractId	diplomeld
firstName	userId	userId	userId	studentId	intitule
lastName	nom_naissance	cfa	employeur_privé_public	companyId	code_rncp
email	nom_usage	denomination_cfa	nom_entreprise	schoolId	niveau
passwordHash	premier_prenom	num_UAI_cfa	numero_adr	nature_contrat	
role	NIR	num_siret_cfa	voie_adr	type_contrat	
connected	numero_adr	numero_adr	complement_adr	emploi_occupe_pendant_contrat	
	voie_adr	voie_adr	code_postal	classification_emploi_conv_collective	
	complement_adr	complement_adr	commune	niveau	
	code_postal	code_postal	tel	coefficient_hierarchique	
	commune	commune	email	duree_periode_essai	
	tel	cfa_lieu_formation	siret	date_debut_contrat	
	email	visa_cfa	type_employeur	date_effet_avenant	
	date_naissance	diplome_vise	employeur_specifique	date_fin_contrat	
	sexe	intitule_diplome	code_NAF	duree_travail_heure	
	departement_naissance	code_diplome	effectif_salaries	duree_travail_minutes	
	commune_naissance	code_rncp	code_IDCC	date_signature_contrat	
	nationalite	date_debut_formation	caisse_retraite_complementaire	salaires_brut_mensuel	
	regime_social	date_fin_formation	organisme_prevoyance	type_derogation	
	sportif	duree_heures_formation	particulier_employeur	num_contrat_precedent	
	rqth	attest_disposer_pieces_justificatives	urssaf	date_debut_formation_employeur	
	situation_avant_contrat	lieu_signature	convention_collective	dangers_exposition_aux_risques	
	dernier_diplome_prepare	signature_employeur	nom_maitre_stage	date_debut_annee_1	
	derniere_classe_annee_suivie	num_declaration_activite	prenom_maitre_stage	date_fin_annee_1	
	intitule_dernier_diplome_prepare	nombre_organisme_formation_intervenant	emploi_maitre_stage	pourcentage_annee_1	
	diplome_obtenu	service_formation_interne	date_naissance_maitre_stage	smic_smc_annee_1	
	avoir_projet_creation_reprise_entreprise	attest_employeur_organisme_formation_dispose_moyen	NIR_maitre_stage	date_debut_annee_2	
	inscrit_pole_emploi	sessionName	email_maitre_stage	date_fin_annee_2	
	num_inscription_pole_emploi		diplome_eleve_obtenu_maitre_stge	pourcentage_annee_2	
	duree_mois_pole_emploi		niveau_eleve_obtenu_maitre_stage	smic_smc_annee_2	
	type_minimum_social_si_beneficiaire			date_debut_annee_3	
				date_fin_annee_3	
				pourcentage_annee_3	
				smic_smc_annee_3	
				date_debut_annee_4	
				date_fin_annee_4	
				pourcentage_annee_4	
				smic_smc_annee_4	
				caisse_retraite_complementaire	
				avantages_nature_nouriture	
				avantages_repas_logement	
				autres_avantages	

3.2 Concevoir et développer une base de données :

Pour ce projet, j'ai conçu ma base de données en adoptant le modèle **Merise**. Ce modèle me permet de structurer efficacement les données tout en assurant une organisation claire et cohérente des informations. Il facilite également la gestion des relations entre les différentes entités de la base, garantissant ainsi une meilleure performance et une évolutivité optimisée pour les besoins futurs du projet.

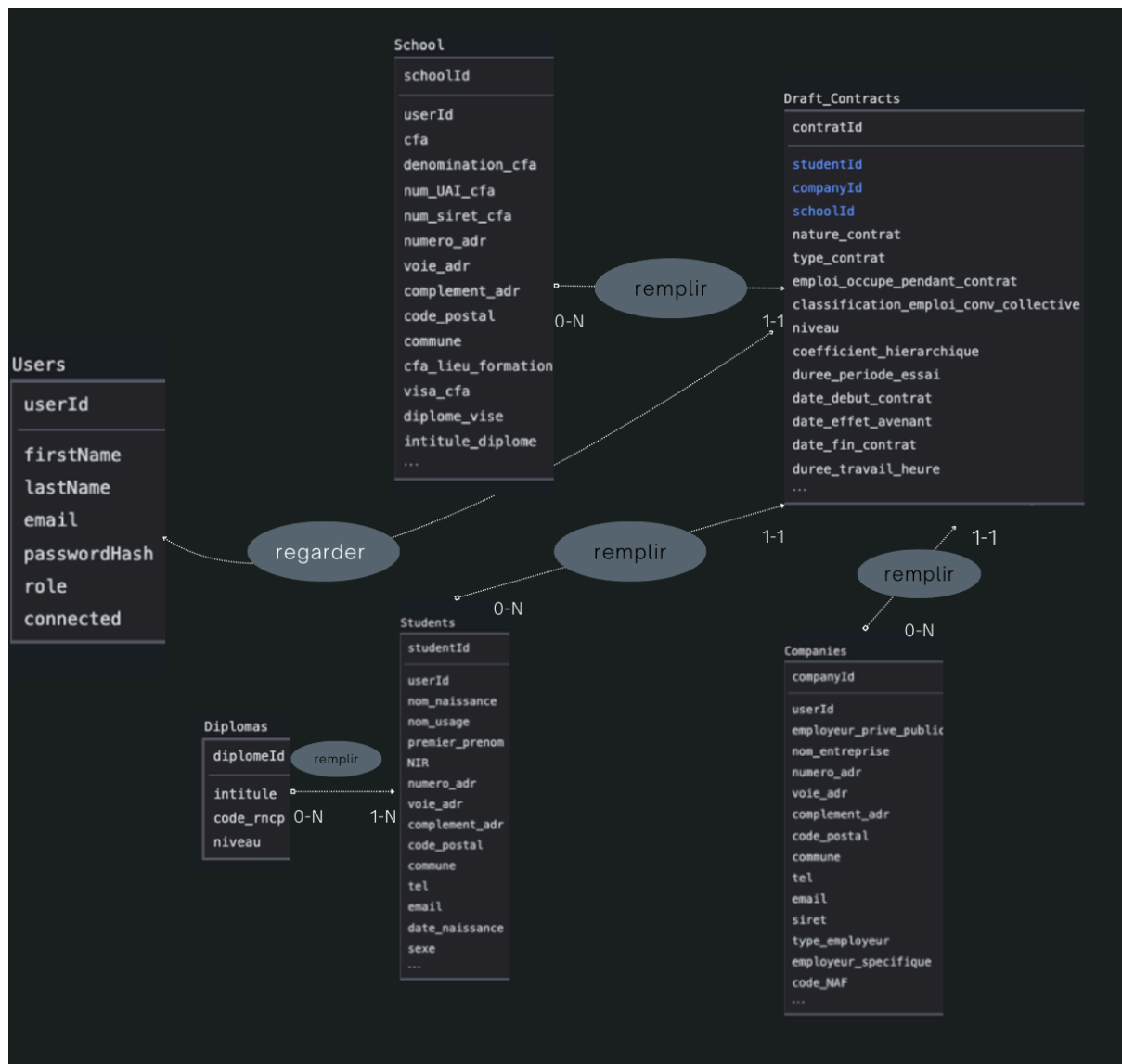
Modèle Conceptuel des Données (MCD)



Le **MCD** représente la première étape de ma modélisation. Il s'agit d'un schéma au plus haut niveau d'abstraction qui me permet de structurer les données indépendamment des contraintes techniques. J'y ai défini les entités principales du système, à savoir Users, Students, School, Companies, et Draft_Contracts, ainsi que leurs attributs respectifs.

- J'ai établi les relations entre ces entités, comme le lien entre Draft_Contracts et les entités Students, Companies, et School. Cela me permet de comprendre quelles informations sont partagées entre ces différentes entités.
- Par exemple, un étudiant (Students) est lié à un contrat spécifique (Draft_Contracts) signé avec une entreprise (Companies) et une institution de formation (School).

Modèle Logique des Données (MLD)



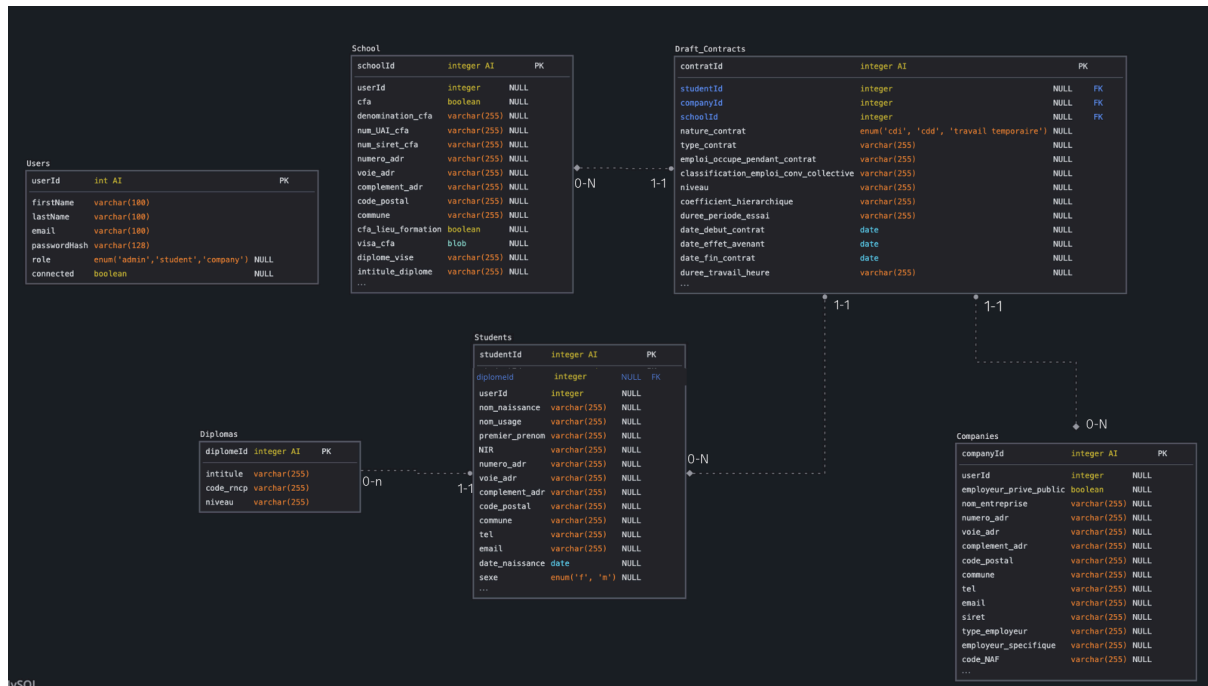
Le **MLD** affine le MCD en y appliquant des règles de normalisation pour s'adapter au modèle relationnel des bases de données. J'y spécifie les relations avec les cardinalités (1-1, 1-N, etc.)

- Dans mon schéma, les relations entre les entités sont décrites par des cardinalités

comme 1-1 ou 0-N, ce qui indique les contraintes d'association entre ces entités.

- Les termes comme "remplir" ou "regarder" dans les relations définissent les interactions possibles entre les tables, par exemple, un étudiant peut remplir un contrat, et une entreprise est associée à un seul contrat.

Modèle Physique des Données (MPD)



Le **MPD** représente la version finale et implémentable du modèle dans le SGBD que j'ai choisi, ici MySQL. Je définis les types de données, les contraintes (clés primaires, clés étrangères) et les relations physiques entre les tables.

J'ai attribué les types de données suivants à chaque attribut en fonction des besoins spécifiques :

- VARCHAR pour les chaînes de caractères (ex : firstName, email dans Users).
- INT pour les nombres entiers, notamment pour les clés primaires et auto-incrémentées (ex : userId, studentId).
- ENUM pour les options prédéfinies, comme dans le cas des rôles dans la table Users (Admin, Student, Company).
- BOOLEAN pour des champs binaires (ex : connected dans Users ou cfa dans School).

Les relations entre les tables sont gérées avec des clés primaires et des clés étrangères :

- userId est la clé primaire de la table Users et une clé étrangère dans les tables Students, School, et Companies, pour relier ces entités à l'utilisateur correspondant.
- studentId, companyId, et schoolId sont des clés étrangères dans la table Draft_Contracts, établissant des relations entre un contrat et un étudiant, une entreprise, ainsi qu'une école.

Chaque table respecte les règles de normalisation pour garantir l'intégrité des données et éviter les redondances inutiles :

- Les clés primaires sont auto-incrémentées (AUTO_INCREMENT) pour assurer l'unicité des enregistrements.
- Les relations sont gérées par des clés étrangères (FOREIGN KEY) afin de garantir l'intégrité référentielle entre les différentes entités.

Les contraintes d'intégrité sont assurées pour éviter les incohérences dans les données :

- Les champs critiques tels que firstName, email et passwordHash dans la table Users sont définis comme NOT NULL, garantissant que ces informations ne peuvent pas être omises lors de la création d'un utilisateur.

Les schémas ne présentent pas tous les éléments car le CERFA contient trop d'informations et il serait illisible de tout inclure dans un seul schéma. J'ai donc préféré montrer la logique globale des relations entre les différentes entités et ensuite détailler précisément ce que chaque table contient à travers le code. Cela permet de comprendre facilement la structure générale tout en garantissant que chaque composant de la base de données est bien pris en compte.

La protection des données personnelles et la RGPD dès la conception

En tant que concepteur de cette base de données, je dois impérativement respecter le Règlement Général sur la Protection des Données (RGPD). Ce règlement impose des obligations strictes quant à la collecte et au traitement des données personnelles. Dans mon cas, cela inclut des informations sensibles comme les noms, adresses e-mails, numéros de téléphone et données de naissance.

Je dois donc veiller à protéger ces données et garantir leur confidentialité. Pour ce faire, j'adopte plusieurs mesures :

- Le chiffrement des mots de passe, comme indiqué par l'attribut passwordHash dans la table Users.
- La limitation de l'accès aux données aux seuls utilisateurs autorisés, en utilisant les rôles définis dans la base (Admin).
- Le respect des droits des utilisateurs, notamment en leur permettant de consulter, modifier ou supprimer leurs données personnelles.

En respectant ces exigences, je m'assure que ma base de données soit conforme aux règles de protection des données personnelles tout en offrant un haut niveau de sécurité.

3.3 Développer des composants d'accès aux données

Afin de vous expliquer les composants d'accès aux données, je vais détailler la manière dont cela s'opère sur notre application en utilisant la méthode CRUD (Create, Read, Update, Delete).

Create :

Dans le processus de création et de collecte des données, nous avons développé une approche méthodique en utilisant MySQL. Plus spécifiquement, nous avons créé plusieurs

tables, chacune correspondant à divers aspects à remplir sur des contrats CERFA distincts, tels que les contrats de Professionnalisation et d'Apprentissage. Sans entrer dans les détails complexes, chaque contrat est divisé en catégories, requérant des contributions respectives de l'école, de l'élève et de l'entreprise. Nous avons structuré notre base de données de manière réfléchie pour garantir une gestion efficace des informations.

Il convient de noter que pour lier notre base de données avec les outils Make et Tally, nous avons procédé au déploiement de notre base de données sur Google Cloud Platform.

Voici comment nous avons initié notre base de données avec deux exemples de table :

- User NAO : Les utilisateurs de l'application
- Students : Les étudiants qui auront à remplir leurs contrats

Explication du code :

- "CREATE DATABASE IF NOT EXISTS NAO;" **création de la base de donnée NAO**
- "CREATE TABLE IF NOT EXISTS Users;" **création de la table Users**
- "userId INT PRIMARY KEY AUTO_INCREMENT" **Création de la colonne UserId qui va se remplir tout seul et qui sera une Clef primaire. Cette clef sera un chiffre**
- "VARCHAR(100) NOT NULL" on oblige à entrer une suite de caractère qui ne peut pas être nulle c'est à dire inexistante

Ici un exemple d'une autre de nos tables : la table Students. Nous retrouvons la même logique que plus haut avec en plus :

- un format de date avec DATE
- définir seulement deux valeurs possibles avec ENUM('F', 'M')
- un booléen avec BOOLEAN qui par défaut sera à 0 c'est à dire négatif (pas sportif ou pas rqth)

Read :

Les utilisateurs bénéficieront d'un accès direct à une visualisation aisée pour attester du remplissage du personnel de l'école, s'occupant de la génération de nos le savons, sont confidentielles. Afin de sécuriser place d'une table avec les bases un utilisateur et

```
CREATE DATABASE IF NOT EXISTS NAO;

USE NAO;

CREATE TABLE IF NOT EXISTS Users (
    userId INT PRIMARY KEY AUTO_INCREMENT,
    firstName VARCHAR(100) NOT NULL,
    lastName VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    passwordHash VARCHAR(128) NOT NULL,
    role ENUM('Admin', 'Student', 'Company'),
    connected BOOLEAN DEFAULT 0
);
```

```
CREATE TABLE IF NOT EXISTS Students (
    studentId INTEGER PRIMARY KEY AUTO_INCREMENT,
    userId INTEGER,
    FOREIGN KEY (userId) REFERENCES Users(userId),
    nom_naissance VARCHAR(255),
    nom_usage VARCHAR(255),
    premier_prenom VARCHAR(255),
    NIR VARCHAR(255),
    numero_adr VARCHAR(255),
    voie_adr VARCHAR(255),
    complement_adr VARCHAR(255),
    code_postal VARCHAR(255),
    commune VARCHAR(255),
    tel VARCHAR(255),
    email VARCHAR(255),
    date_naissance DATE,
    sexe ENUM('F', 'M'),
    departement_naissance VARCHAR(255),
    commune_naissance VARCHAR(255),
    nationalite ENUM('1', '2', '3'),
    regime_social ENUM('1', '2'),
    sportif BOOLEAN DEFAULT 0,
    rqth BOOLEAN DEFAULT 0,
    situation_avant_contrat VARCHAR(255),
    dernier_diplome_prepare VARCHAR(255),
    derniere_classe_annee_suivie VARCHAR(255),
    intitule_dernier_diplome_prepare VARCHAR(255),
    diplome_obtenu VARCHAR(255),
    avoir_projet_creation_reprise_entreprise BOOLEAN DEFAULT 0
);
```

```

@app.route('/students')
def allStudent():
    # Connecting to the database
    cur = mysql.connection.cursor()

    # Executing SQL query to retrieve students
    cur.execute("SELECT * FROM Students")

    # Retrieving query results
    students = cur.fetchall()

    # Closing the cursor
    cur.close()

    # Rendering HTML template with results
    return render_template('students.html', students=students)

```

Ci-dessus un exemple de lien entre notre base de données et notre application.

Dans cet exemple, la route /students est définie avec la fonction allStudent(). Cela signifie que lorsque l'URL /students est accédée dans le navigateur, la fonction allStudent() sera exécutée.

À l'intérieur de cette fonction :

- Nous nous connectons à la base de données MySQL en utilisant `mysql.connection.cursor()`.
- Nous exécutons une requête SQL pour récupérer tous les étudiants de la table Students en utilisant `cur.execute("SELECT * FROM Students")`.
- Nous récupérons les résultats de la requête à l'aide de `cur.fetchall()`, qui retourne une liste de tuples où chaque tuple représente un enregistrement de la table.
- Nous fermons le curseur de la base de données avec `cur.close()` pour libérer les ressources.
- Enfin, nous renvoyons les résultats sous forme de modèle HTML en utilisant `render_template('students.html', students=students)`. Cela signifie que nous rendons le modèle `students.html` et lui passons la liste des étudiants récupérés de la base de données, qui seront accessibles dans le modèle sous le nom `students`.

Ainsi, lorsque cette route est accédée dans un navigateur, elle retournera une page HTML contenant les détails de tous les étudiants stockés dans la base de données.

Update :

Les administrateurs disposent d'un accès direct à la base de données via google cloud platform pour effectuer des modifications sur les informations saisies.

Ce morceau de code définit une route API qui permet de mettre à jour les attributs d'un utilisateur dans la base de données via une requête HTTP de type PUT.

```
@app.route('/api/v1/user/<email>', methods=['PUT'])
def modify_user(email):
    result = users.update_attribute(email, request.form.to_dict())
    if result:
        return jsonify(status="True",
                        message= "updated",
                        result={
                            "nom": result.lastName,
                            "prenom": result.firstName,
                            "email": result.email,
                            "mot de passe": result.passwordHash,
                            "rôle": result.role,
                            "connecté": result.connected}
                        )
    return jsonify(status= "False")
```

Voici un résumé de son fonctionnement en termes de mise à jour de la base de données :

- **Route** : `@app.route('/api/v1/user/<email>', methods=['PUT'])`
Cette route accepte les requêtes PUT à l'URL `/api/v1/user/<email>`. Le paramètre dynamique `<email>` représente l'identifiant de l'utilisateur à modifier.
- **Fonction** : `modify_user(email)`
Cette fonction récupère les nouvelles données pour l'utilisateur via `request.form.to_dict()`, qui convertit les données du formulaire en dictionnaire. Ensuite, elle appelle la méthode `users.update_attribute(email, ...)` pour mettre à jour les attributs de l'utilisateur identifié par son email dans la base de données.
- **Si la mise à jour réussit** :
Si `users.update_attribute()` renvoie un résultat positif (mise à jour réussie), la fonction retourne une réponse JSON indiquant que l'opération a été effectuée avec succès. Les informations mises à jour de l'utilisateur sont renvoyées sous forme de JSON :
 - Nom
 - Prénom
 - Email
 - Mot de passe (haché)
 - Rôle
 - Statut de connexion
- **Si la mise à jour échoue** :
Si la mise à jour échoue, la fonction renvoie un message d'échec avec le statut "False".

Delete

Enfin, afin de respecter les normes rigoureuses du règlement général sur la protection des données (RGPD), nous donnerons l'accès à la base de données pour supprimer les données via notre application cinq ans après la sortie de l'élève de l'école. Il sera possible, directement sur GCP de supprimer les données qui ne sont plus utiles

Il y a également la possibilité de supprimer les données directement via l'application

```
@app.route('/api/v1/user/<email>', methods=['DELETE'])
def delete_user(email):
    result = users.delete_user_by_id(email)
    if result:
        return jsonify(status="True",
                        message= "Deleted",
                        email=email
                        )
    return jsonify(status="False")
```

Cette route supprime un utilisateur de la base de données en fonction de son email. Si l'opération est réussie, elle renvoie un message de confirmation avec l'email supprimé. En cas d'échec, elle renvoie un statut d'échec.

4. Le front-end

Afin de faciliter au maximum la tâche des administrateurs, nous avons décidé de concevoir une interface axée sur la simplicité et l'efficacité. Notre objectif principal est de permettre à notre application de maximiser le gain de temps et, par conséquent, d'optimiser les ressources financières de l'entreprise qui nous a confié la réalisation de ce projet. L'accent est mis sur la fonctionnalité plutôt que sur le design, dans le but de répondre aux besoins opérationnels essentiels de manière pragmatique et efficiente.

Le développement de la partie front-end de l'interface utilisateur web pour l'école Beauvoir de formation en data science. L'interface inclut des fonctionnalités pour la gestion des sessions, des informations sur les étudiants, les diplômes, les entreprises, et sur l'école. Le code HTML, CSS, et Python (Flask) sont utilisés pour construire cette interface.

4.1 Voici les différentes pages HTML :

- base.html : Un fichier de base utilisé comme modèle (template) pour d'autres pages HTML. Contient souvent des éléments communs comme le header, le footer ou la structure

générale du site.

- companies.html : Affiche des informations sur les entreprises partenaires ou les sociétés associées à l'application.
- formations.html : Affiche des formations
- from_tally.html : affichage du formulaire tally
- home.html : Page d'accueil de l'application ou du site web.
- index.html : Une page principale d'index.
- schoolCreateSession.html : Page permettant la création de sessions pour une école.
- schoolInformation.html : Affiche des informations détaillées sur l'école.
- sessions.html : Affiche les sessions disponibles ou passées pour une école ou une formation.
- signIn.html : Page de connexion pour les utilisateurs. Permet aux utilisateurs d'accéder à leur compte en entrant leur identifiant et mot de passe.
- signUp.html : Page d'inscription. Permet aux nouveaux utilisateurs de créer un compte en fournissant les informations nécessaires.
- students.html : Affiche la liste des étudiants inscrits dans une école ou une formation. Cela pourrait aussi inclure des détails spécifiques à chaque étudiant.
- viewPdf.html : Page permettant d'afficher des documents PDF, liés aux Cerfa

4.2 Création de la Structure HTML de Base (base.html)

Définition de la structure de base du document HTML.

Inclusion des métadonnées nécessaires (titre, description, favicon).

Intégration des liens vers les fichiers CSS et les bibliothèques externes (Bootstrap, Bootstrap Icons).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>NAO-School Formation | Data Engineer | Data Scientist | DataAnalyst | Big Data</title>
  <link rel="icon" type="image/x-icon" href="/static/img/favicon.png">
  <meta name="description" content="Notre formation Nao data science vous permettra de démarrer une carrière dans la">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet" integrity="s">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
  <link rel="stylesheet" href="../static/css/style.css">
</head>
```

- <!DOCTYPE html> : Déclaration du type de document HTML.
- <html lang="en"> : Début du document HTML avec l'attribut lang pour spécifier la langue.
- <head> : Contient les métadonnées, les liens vers les fichiers CSS, et le titre de la page.
 - <meta charset="UTF-8"> : Définit l'encodage des caractères.
 - <meta http-equiv="X-UA-Compatible" content="IE=edge"> : Assure la compatibilité avec Internet Explorer.
 - <meta name="viewport" content="width=device-width, initial-scale=1.0"> : Assure la mise en page réactive.
 - <title> : Définit le titre de la page.
 - <link rel="icon" type="image/x-icon" href="/static/img/favicon.png"> : Définit l'icône de la page.
 - <meta name="description" content="..."> : Définit la description de la page.
 - <link href="..." rel="stylesheet"> : Intègre les fichiers CSS de Bootstrap et les icônes Bootstrap.
 - <link rel="stylesheet" href="../static/css/style.css"> : Intègre le fichier CSS personnalisé.

4.3 Création de l'En-tête (Header)



Home Sessions School Students Diplomas Companies

Ajouter un en-tête avec le logo de l'école et un menu de navigation.

Inclure des liens vers les différentes sections de l'application (Home, Sessions, School, Students, Diplomas, Companies).

Ajouter des boutons pour la connexion et l'inscription.

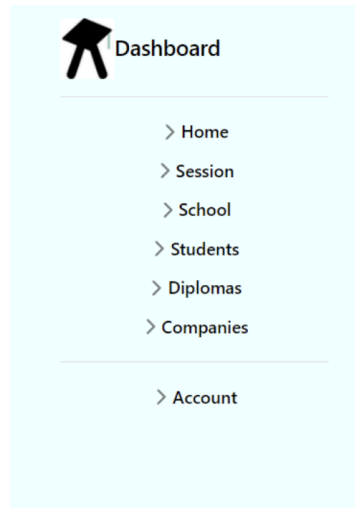
```
<!-- Header -->
<header>
  <div class="container">
    <header class="d-flex flex-wrap align-items-center justify-content-center justify-content-md-between py-3 mb-4 border-bottom">
      <a href="/" class="d-flex align-items-center col-md-3 mb-2 mb-md-0 text-dark text-decoration-none">
        
      </a>

      <ul class="nav col-12 col-md-auto mb-2 justify-content-center mb-md-0">
        <li><a href="{{ url_for('home') }}" class="nav-link px-2 link-secondary">Home</a></li>
        <li><a href="{{ url_for('allSessions') }}" class="nav-link px-2 link-dark">Sessions</a></li>
        <li><a href="{{ url_for('allSessionsSchool') }}" class="nav-link px-2 link-dark">School</a></li>
        <li><a href="{{ url_for('suivre le lien (cmd + clic) ')" class="nav-link px-2 link-dark">Students</a></li>
        <li><a href="{{ url_for('allFormation') }}" class="nav-link px-2 link-dark">Diplomas</a></li>
        <li><a href="{{ url_for('allCompanies') }}" class="nav-link px-2 link-dark">Companies</a></li>
      </ul>

      <div class="col-md-3 text-end">
        <!-- <button type="button" class="btn btn-outline-primary me-2">Login</button> -->
        <a class="btn btn-outline-primary me-2" href="{{ url_for('signIn') }}" role="button">Sign-in</a>
        <a class="btn btn-outline-primary" href="{{ url_for('signUp') }}" role="button">Sign-up</a>
      </div>
    </header>
  </div>
</header>
<main>
```

- `<header>` : Début de la section d'en-tête.
- `<div class="container">` : Conteneur Bootstrap pour centrer le contenu.
- `<header class="d-flex flex-wrap align-items-center justify-content-center justify-content-md-between py-3 mb-4 border-bottom">` : En-tête avec des classes Bootstrap pour la mise en page et le style.
- `<a href="/" class="d-flex align-items-center col-md-3 mb-2 mb-md-0 text-dark text-decoration-none">` : Lien vers la page d'accueil avec le logo de l'école.
- `<ul class="nav col-12 col-md-auto mb-2 justify-content-center mb-md-0">` : Liste de navigation avec des liens vers les différentes sections.
- `<li><a href="{{ url_for('home') }}" class="nav-link px-2 link-secondary">Home</a></li>` : Lien vers la page d'accueil.
- `<div class="col-md-3 text-end">` : Conteneur pour les boutons de connexion et d'inscription.
- `<a class="btn btn-outline-primary me-2" href="{{ url_for('signIn') }}" role="button">Sign-in</a>` : Bouton de connexion.
- `<a class="btn btn-outline-primary" href="{{ url_for('signUp') }}" role="button">Sign-up</a>` : Bouton d'inscription.

4.4 Création de la Barre Latérale



Ajouter une barre latérale pour la navigation secondaire.

Inclure des sections pour Home, Session, School, Students, Diplomas, Companies, et Account.

```
<aside>
  <div class="flex-shrink-0 p-3" style="width: 280px;">
    <a href="/" class="d-flex align-items-center pb-3 mb-3 link-body-emphasis text-decoration-none border-bottom">
      
      <span class="fs-5 fw-semibold">Dashboard</span>
    </a>
    <ul class="list-unstyled ps-0">
      <li class="mb-1">
        <button class="btn btn-toggle d-inline-flex align-items-center rounded border-0 collapsed" data-bs-toggle="collapse" data-bs-target="#home-collapse">
          Home
        </button>
        <div class="collapse" id="home-collapse">
          <ul class="btn-toggle-nav list-unstyled fw-normal pb-1 small">
            <li><a href="{{ url_for('home') }}" class="link-body-emphasis d-inline-flex text-decoration-none rounded">About</a></li>
          </ul>
        </div>
      </li>
      <!-- Autres sections similaires -->
    </ul>
  </div>
</aside>
```

- `<aside>` : Début de la section de la barre latérale.
- `<div class="flex-shrink-0 p-3" style="width: 280px;">` : Conteneur pour la barre latérale avec une largeur fixe.
- `<a href="/" class="d-flex align-items-center pb-3 mb-3 link-body-emphasis text-decoration-none border-bottom">` : Lien vers la page d'accueil avec l'icône de l'école et le texte "Dashboard".
- `<ul class="list-unstyled ps-0">` : Liste non stylisée pour les sections de la barre latérale.
- `<li class="mb-1">` : Élément de liste pour une section.
- `<button class="btn btn-toggle d-inline-flex align-items-center rounded border-0 collapsed" data-bs-toggle="collapse" data-bs-target="#home-collapse" aria-expanded="false">` : Bouton pour afficher/masquer la section "Home".
- `<div class="collapse" id="home-collapse">` : Conteneur pour le contenu de la section "Home".
- `<ul class="btn-toggle-nav list-unstyled fw-normal pb-1 small">` : Liste pour les sous-sections de "Home".
- `<li><a href="{{ url_for('home') }}" class="link-body-emphasis d-inline-flex text-decoration-none rounded">About</a></li>` : Lien vers la page "About".

4.5 Création du Contenu Dynamique

Ajouter une section pour le contenu dynamique qui sera rempli par les templates Flask.

```
<div class="col-9">
  {% block message %}
  {% endblock %}
  {% block title %}
  {% endblock %}
  {% block content %}
  {% endblock %}
</div>
```

- `<div class="col-9">` : Conteneur pour le contenu dynamique.
- `{% block message %}{% endblock %}` : Bloc pour les messages dynamiques.
- `{% block title %}{% endblock %}` : Bloc pour le titre dynamique.
- `{% block content %}{% endblock %}` : Bloc pour le contenu dynamique.

4.6 Création du Pied de Page (Footer)



© 2024 par Beauvoir, tous droits réservés.



Ajouter un pied de page avec des informations de contact et des liens vers les réseaux sociaux.

```
<!-- Footer -->
<div class="container">
  <div class="d-flex flex-wrap justify-content-between align-items-center py-3 my-4 border-top">
    <div class="col-md-4 d-flex align-items-center">
      <a href="/" class="mb-3 me-2 mb-md-0 text-body-secondary text-decoration-none lh-1">
        <!-- <svg class="bi" width="30" height="24"><use xlink:href="#bootstrap"></use></svg> -->
        
      </a>
      <span class="mb-3 mb-md-0 text-body-secondary">© 2023 Company, Inc</span>
    </div>
    <ul class="nav col-md-4 justify-content-end list-unstyled d-flex">
      <li class="ms-3"><a class="text-body-secondary" href="#"><svg class="bi" width="24" height="24"><use xlink:href="#twitter"></use></svg></a></li>
      <li class="ms-3"><a class="text-body-secondary" href="#"><svg class="bi" width="24" height="24"><use xlink:href="#instagram"></use></svg></a></li>
      <li class="ms-3"><a class="text-body-secondary" href="#"><svg class="bi" width="24" height="24"><use xlink:href="#facebook"></use></svg></a></li>
    </ul>
  </div>
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js" integrity="sha384-C6RzsynM9kWD rHNeT87bh950GyZPhcTNXj1NW7RuBCsyN/
</body>
</html>
```

- `<div class="d-flex flex-wrap justify-content-between align-items-center py-3 my-4 border-top">` : Pied de page avec des classes Bootstrap pour la mise en page et le style.
- `<div class="col-md-4 d-flex align-items-center">` : Conteneur pour les informations de contact.
- `<a href="/" class="mb-3 me-2 mb-md-0 text-body-secondary text-decoration-none lh-1">` : Lien vers la page d'accueil avec l'icône de l'école.
- `<span class="mb-3 mb-md-0 text-body-secondary">© 2023 Company, Inc</span>` : Texte des droits d'auteur.
- `<ul class="nav col-md-4 justify-content-end list-unstyled d-flex">` : Liste pour les liens vers les réseaux sociaux.
- `<li class="ms-3"><a class="text-body-secondary" href="#"><svg class="bi" width="24" height="24"><use xlink:href="#twitter"></use></svg></a></li>` : Lien vers Twitter avec une icône.
- `<li class="ms-3"><a class="text-body-secondary" href="#"><svg class="bi" width="24"`

- height="24"><use xlink:href="#instagram"></use></svg> : Lien vers Instagram avec une icône.
- <li class="ms-3"><svg class="bi" width="24" height="24"><use xlink:href="#facebook"></use></svg> : Lien vers Facebook avec une icône.

4.7 CSS pour styliser la page

Ajouter des styles pour améliorer l'apparence de l'interface.

```
body {
  min-height: 100vh;
}

main {
  height: 100vh;
  max-height: 100vh;
  overflow-x: auto;
}

.dropdown-toggle { outline: 0; }

.btn-toggle {
  padding: .25rem .5rem;
  font-weight: 600;
  color: var(--bs-emphasis-color);
  background-color: transparent;
}

.btn-toggle:hover,
.btn-toggle:focus {
  color: rgba(var(--bs-emphasis-color-rgb), .85);
  background-color: var(--bs-tertiary-bg);
}

.btn-toggle::before {
  width: 1.25em;
  line-height: 0;
  content: url("data:image/svg+xml,%3csvg xmlns='http://www.w3.org/2000/svg' wid
  transition: transform .35s ease;
  transform-origin: .5em 50%;
}

[data-bs-theme="dark"] .btn-toggle::before {
  content: url("data:image/svg+xml,%3csvg xmlns='http://www.w3.org/2000/svg' wid
}

.btn-toggle[aria-expanded="true"] {
  color: rgba(var(--bs-emphasis-color-rgb), .85);
}
```

```

}

.btn-toggle-nav a {
  padding: .1875rem .5rem;
  margin-top: .125rem;
  margin-left: 1.25rem;
}

.btn-toggle-nav a:hover,
.btn-toggle-nav a:focus {
  background-color: var(--bs-tertiary-bg);
}

.scrollarea {
  overflow-y: auto;
}

.video {
  height: 300px;
  width: 520px;
  align-content: center;
  align-items: center;
}

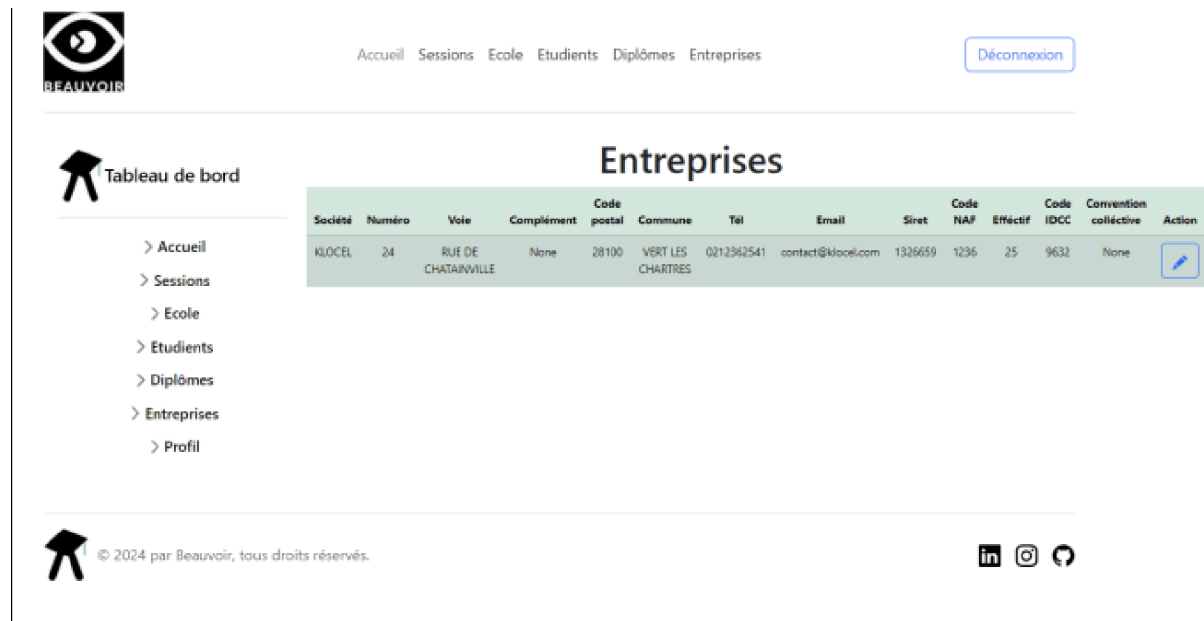
table {
  font-size: 12px;
}

```

Explication :

- `body { min-height: 100vh; }` : Assure que le corps de la page occupe au moins la hauteur de la fenêtre.
- `main { height: 100vh; max-height: 100vh; overflow-x: auto; }` : Assure que la section principale occupe la hauteur de la fenêtre et permet le défilement horizontal.
- `.dropdown-toggle { outline: 0; }` : Supprime le contour des boutons de menu déroulant.
- `.btn-toggle { ... }` : Styles pour les boutons de basculement.
- `.btn-toggle:hover, .btn-toggle:focus { ... }` : Styles pour les boutons de basculement au survol et au focus.
- `.btn-toggle::before { ... }` : Styles pour l'icône de flèche des boutons de basculement.
- `[data-bs-theme="dark"] .btn-toggle::before { ... }` : Styles pour l'icône de flèche en mode sombre.
- `.btn-toggle[aria-expanded="true"] { ... }` : Styles pour les boutons de basculement lorsqu'ils sont étendus.
- `.btn-toggle[aria-expanded="true"]::before { ... }` : Styles pour l'icône de flèche lorsqu'elle est étendue.
- `.btn-toggle-nav a { ... }` : Styles pour les liens de navigation dans les boutons de basculement.
- `.btn-toggle-nav a:hover, .btn-toggle-nav a:focus { ... }` : Styles pour les liens de navigation au survol et au focus.
- `.scrollarea { overflow-y: auto; }` : Assure le défilement vertical dans les zones de défilement.
- `.video { ... }` : Styles pour les vidéos.
- `table { font-size: 12px; }` : Styles pour les tableaux.

Exemple de l'affichage des datas des entreprises :



Langages et Technologies Utilisés

HTML

Le langage **HTML** a été utilisé pour structurer les pages web. Il permet de définir la hiérarchie et la sémantique des différents éléments de la page, comme les titres, paragraphes, images, liens, etc. C'est le socle de toute page web, assurant une bonne accessibilité et lisibilité du contenu pour les utilisateurs et les moteurs de recherche.

CSS

Le **CSS** a été employé pour styliser les pages web. Il permet de définir l'apparence visuelle des éléments HTML, en contrôlant la disposition, les couleurs, les polices, les marges, et bien plus encore. Les styles sont centralisés dans un fichier CSS (style.css), garantissant une meilleure maintenabilité et une séparation claire entre la structure HTML et le style visuel.

Bootstrap

Le framework **Bootstrap** a été utilisé pour faciliter le développement d'une interface utilisateur moderne et responsive. Grâce à son système de grille, il permet de structurer facilement les mises en page complexes en utilisant des **containers**, **rows**, et **colonnes**. En plus de la grille, Bootstrap propose une large gamme de composants préconçus, comme des boutons, des barres de navigation, et des formulaires, permettant d'accélérer considérablement le développement.

Classes Bootstrap

Les **classes Bootstrap** prédéfinies telles que container, row, col, nav, et btn ont été utilisées pour structurer et styliser les éléments HTML sans avoir à écrire de styles CSS supplémentaires. Ces classes facilitent l'agencement des éléments et assurent une apparence cohérente et professionnelle, tout en permettant une personnalisation via CSS si nécessaire.

5. LE BACK END

5.1 Partie récolte des données

À la demande de l'entreprise, la collecte des données s'effectue au moyen de technologies no-code et low-code. Pour mener à bien cette collecte, nous avons recours aux technologies Make et Tally.

Création de formulaires :

Afin de collecter efficacement les données, il a été nécessaire de créer quatre formulaires :

- Formulaire pour les données fixes de l'école : Ce formulaire est destiné à recueillir les informations de l'école qui sont susceptibles de rester constantes ou presque au fil des sessions (SIREN, nom de l'école, adresse, etc.). Bien entendu, ces données restent accessibles pour d'éventuelles mises à jour majeures.

School_information_CERFA

L'école s'appelle SIRET

Le CFA est le lieu de formation principal est-ce un CFA d'entreprise ?

Le numéro UAI est

Le CFA est au

Le numéro de déclaration d'activité de l'école est

Le nombre d'organisme(s) de formation qui interviennent est de

☐ J'atteste disposer de l'ensemble des pièces justificatives

Il ne s'agit pas d'un service de formation interne

Cachet de l'école Signature de la directrice

Lieu de signature

- Formulaire pour les informations relatives aux sessions : Ce formulaire permet de collecter les détails des sessions qui peuvent varier selon les classes (date, horaires de formation, etc.).

School_create_session

Nom de la session :

Le **code RNCP** est

L'**intitulé** précis du **RNCP** est

Le **code du diplôme** est

L'action de formation **démarre** prend **fin** le

Nombre **d'heures** de la formation

dont heures dédiées aux enseignements **généraux, professionnels et technologiques**

540 alternance, 402 bootcamp, 160 contrat professionnalisation signé après le bootcamp, 400 POEI

Le code de spécialité de formation est

Code NFS à 3 chiffres. Consultable dans le document [Nomenclature des formations \(CNIS\)](#)

Le type de qualification, visée est

L'équivalence du **diplôme** ou **titre visé** est

- Formulaire pour les informations des étudiants : Ce formulaire est conçu pour rassembler les données personnelles et académiques des étudiants.

Contrat_CERFA_Etudiant

Afin de réaliser vos contrats, nous avons besoin d'informations légales

Je m'appelle

J'ai ans et ma date de naissance est

Mon genre déclaré est ☐ Un Homme ☐ Une femme

☐ Je ne suis pas une personne en situation de handicap

☐ Je ne suis pas une personne figurante sur la liste des sportifs de haut niveau

☐ Je n'envisage pas de créer ou reprendre une société nécessitant l'obtention d'un diplôme

Mon numéro de sécurité sociale est

Comment trouver mon numéro de Sécurité Sociale sur ma carte vitale



Mes coordonnées

J'habite au

Complément d'adresse si le nombre de caractères est dépassé dans la rubrique rue/résidence

Ma pièce d'identité

Limite de taille : 10 Mo

Mon CV

Limite de taille : 10 Mo

L'e-mail de l'entreprise

- Formulaire pour les informations des entreprises : Ce formulaire vise à recueillir les informations pertinentes concernant les entreprises associées ou partenaires.

Cerfa Entreprise Siren

Pour faciliter le remplissage du questionnaire qui nous permettra de compléter les contrats CERFA de votre futur collaborateur, nous avons besoin des informations suivantes de votre futur collaborateur nous avons besoin de ces quelques informations :

Le nom de l'entreprise

Le numéro de SIREN

L'étudiant recherche un

Chacun de ces formulaires a été élaboré pour simplifier la gestion et l'actualisation des données, en assurant une collecte structurée et centralisée des informations essentielles.

RGPD : Pour garantir la conformité avec le RGPD, nous avons intégré une case à cocher dans chacun de nos formulaires de collecte de données. Cette case stipule : **"Vous autorisez l'école, dans le cadre de l'élaboration du contrat CERFA, à utiliser vos données."**

Cette démarche vise à assurer que les utilisateurs sont pleinement informés et donnent leur consentement éclairé concernant l'utilisation de leurs données personnelles. Nous nous engageons à traiter ces données avec le plus grand soin et à respecter les droits des utilisateurs conformément à la législation sur la protection des données.

Lien avec notre base de données :

Afin de répondre aux demandes de la Nao, notre base de données a été migrée vers Google Cloud Platform (GCP) afin de l'intégrer à Make, un outil de développement low code. Cette démarche nous a permis de maximiser les fonctionnalités de Make tout en bénéficiant de la robustesse et de la fiabilité de l'infrastructure de GCP.

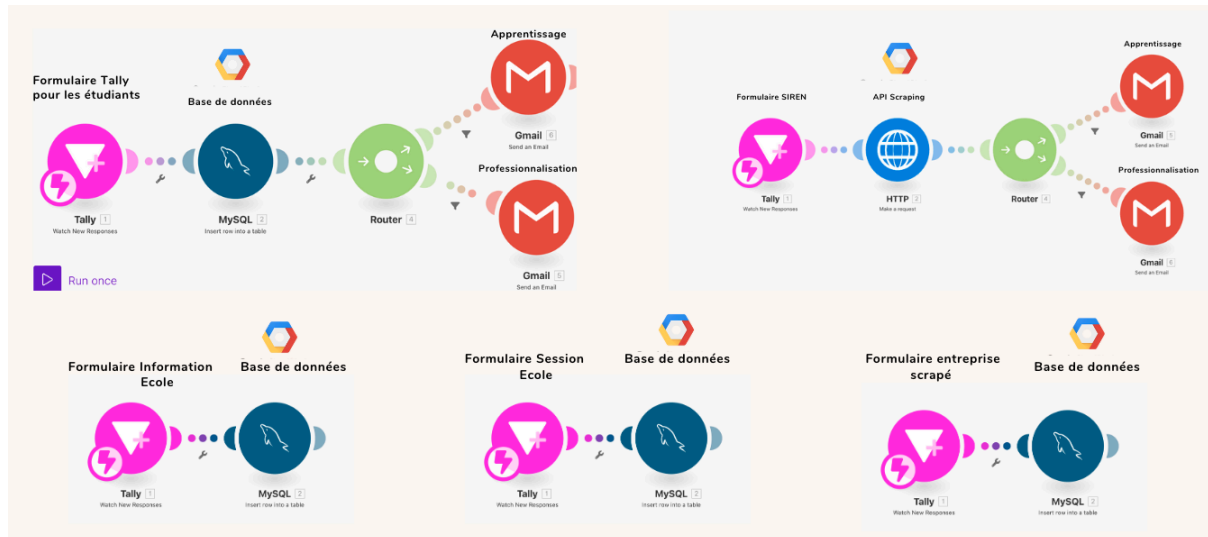
Les scénarios permettant de connecter nos formulaires avec nos bases de données sur Make ont été élaborés et mis en place avec succès. Cette intégration a facilité l'utilisation et la manipulation des données de manière efficace et conviviale, renforçant ainsi notre capacité à développer rapidement des solutions tout en maintenant la cohérence et la qualité des données.

Parallèlement, nous avons également réalisé une intégration similaire avec l'API de scraping. Cette démarche automatisée nous a permis de collecter des données à partir de différentes sources en ligne de manière efficace et fiable, enrichissant ainsi notre capacité à fournir des informations précieuses à nos utilisateurs.

En combinant nos bases de données sur GCP avec Make et l'API de scraping, nous avons consolidé notre infrastructure technologique et renforcé notre agilité dans le développement

et la fourniture de solutions innovantes. Ce projet a été un succès et a contribué à répondre aux besoins changeants de notre entreprise

Voici les différents scénarios de make expliqués juste au dessus :



5.2 Mise en oeuvre

a. Configuration de l'environnement

- Utilisation du framework Flask pour le développement de la partie back-end, en lien avec une base de données MySQL.
- Utilisation de dotenv pour charger les variables d'environnement depuis un fichier .env. Cela permet de sécuriser des informations sensibles comme les identifiants de la base de données (host, user, password) en ne les codant pas en dur dans l'application.

```
from dotenv import load_dotenv, find_dotenv
import os

app = Flask(__name__)
load_dotenv(find_dotenv())

# MySQL database configuration
app.config['MYSQL_HOST'] = os.environ.get("MYSQL_HOST")
app.config['MYSQL_USER'] = os.environ.get("MYSQL_USER")
app.config['MYSQL_PASSWORD'] = os.environ.get("MYSQL_PASSWORD")
app.config['MYSQL_DB'] = os.environ.get("MYSQL_DB")
```

Les informations sensibles comme MYSQL_HOST, MYSQL_USER, et MYSQL_PASSWORD ne sont jamais directement exposées dans le code source, renforçant ainsi la sécurité.

- `from dotenv import load_dotenv, find_dotenv` : Cela importe deux fonctions de la bibliothèque `dotenv`. `find_dotenv` trouve automatiquement le fichier `.env`, et `load_dotenv` charge les variables d'environnement définies dans ce fichier.
- `import os` : Permet d'interagir avec les variables d'environnement du système d'exploitation.
- `app = Flask(name)` : Initialise une instance de l'application Flask, le cœur de l'application.
- `load_dotenv(find_dotenv())` : Cette ligne charge les variables d'environnement contenues dans le fichier `.env`, afin que celles-ci puissent être utilisées dans l'application.
- `app.config['MYSQL_HOST']` : Récupère la variable d'environnement `MYSQL_HOST` et la stocke dans la configuration de l'application Flask pour définir l'adresse du serveur MySQL.
- `app.config['MYSQL_USER']` : Récupère la variable d'environnement `MYSQL_USER`, définissant l'utilisateur MySQL.
- `app.config['MYSQL_PASSWORD']` : Récupère la variable d'environnement `MYSQL_PASSWORD`, correspondant au mot de passe de la base de données MySQL.
- `app.config['MYSQL_DB']` : Récupère la variable d'environnement `MYSQL_DB`, définissant le nom de la base de données utilisée.

b. Gestion des utilisateurs et sessions

Des routes ont été créées pour la gestion de l'authentification, notamment pour la connexion et l'inscription des utilisateurs.

Bcrypt a été utilisé pour hacher les mots de passe avant de les stocker dans la base de données, une pratique essentielle pour assurer la protection des données des utilisateurs.

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        # Retrieving form data
        nom = request.form['nom']
        prenom = request.form['prenom']
        email = request.form['email']
        mot_de_passe = request.form['mot_de_passe']
        role = request.form['role']

        # Hashing the password using bcrypt
        mot_de_passe_hache = bcrypt.hashpw(mot_de_passe.encode('utf-8'), bcrypt

        # Inserting the user into the database
        cur = mysql.connection.cursor()
        cur.execute("INSERT INTO utilisateurs (nom, prenom, email, mot_de_passe_hache, role)
                    (nom, prenom, email, mot_de_passe_hache, role)")
        mysql.connection.commit()
        cur.close()
        return redirect(url_for('home'))

    return render_template('register.html')
```

- **@app.route('/register', methods=['GET', 'POST'])** : Cette ligne définit une route pour l'enregistrement des utilisateurs. La méthode POST est utilisée pour envoyer des données (formulaire), et GET pour afficher la page d'inscription.
- **if request.method == 'POST'** : Vérifie si la méthode HTTP utilisée est POST, indiquant que des données sont soumises via un formulaire.
- **request.form['...']** : Récupère les données saisies dans le formulaire d'inscription, telles que le nom, le prénom, l'email, le mot de passe et le rôle de l'utilisateur.
- **bcrypt.hashpw(...)** : Hache le mot de passe utilisateur en utilisant l'algorithme bcrypt. Cela garantit que le mot de passe est stocké de manière sécurisée dans la base de données.
- **mysql.connection.cursor()** : Ouvre un curseur pour exécuter des requêtes SQL.
- **cur.execute(...)** : Exécute une requête SQL qui insère les données de l'utilisateur (nom, prénom, email, mot de passe haché, rôle) dans la table utilisateurs.
- **mysql.connection.commit()** : Valide la transaction pour s'assurer que les modifications sont enregistrées dans la base de données.
- **cur.close()** : Ferme le curseur après exécution de la requête, libérant ainsi les ressources.
- **redirect(url_for('home'))** : Redirige l'utilisateur vers la page d'accueil après l'inscription.

c. Gestion des données avec MySQL

L'application interagit avec la base de données MySQL pour récupérer ou insérer des informations. Ici un exemple d'une route permettant de récupérer les informations de tous les étudiants et de les afficher sur une page HTML.

```
@app.route('/students')
def allStudent():
    # Connecting to the MySQL database
    cur = mysql.connection.cursor()

    # Executing SQL query to retrieve all students
    cur.execute("SELECT * FROM Students")

    # Fetching all results from the query
    students = cur.fetchall()

    # Closing the cursor
    cur.close()

    # Rendering the HTML template with the results
    return render_template('students.html', students=students)
```

- **@app.route('/students')** : Crée une route /students qui sera appelée pour récupérer et afficher la liste des étudiants.
- **mysql.connection.cursor()** : Ouvre un curseur pour exécuter une requête SQL afin de récupérer les données de la base de données.
- **cur.execute("SELECT * FROM Students")** : Exécute une requête SQL qui récupère toutes les données de la table Students.
- **cur.fetchall()** : Récupère tous les résultats de la requête SQL et les stocke dans la variable students.
- **cur.close()** : Ferme le curseur après avoir récupéré les données pour libérer les ressources.

- `render_template('students.html', students=students)` : Rend la page HTML `students.html` en passant les données récupérées (`students`) pour qu'elles soient affichées dans la vue.

d. Génération et affichage du contrat

L'application permet de générer et d'afficher des contrats sous forme de fichiers PDF. Ces fichiers sont remplis automatiquement avec les données récupérées de la base de données.

```
@app.route('/printcontract/<id>')
def printContract(id):
    # Fetch student data from the database
    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM students WHERE studentId=%s", (id,))
    student = cur.fetchall()

    # Fetch school data
    cur.execute("SELECT * FROM school")
    school = cur.fetchall()

    # Fetch company data based on student's company email
    cur.execute("SELECT * FROM companies WHERE email=%s", (student[0]['company_email'],))
    company = cur.fetchall()

    # Prepare data for PDF generation
    datas = {'student': student[0], 'school': school[0], 'company': company[0]}
    contract = student[0]['type_contrat']

    # Generate PDF contract
    list_champs, doc = fill_data_pdf(contract)
    update_datas = recup_data(datas)
    update_data_pdf(list_champs, doc, update_datas, contract)

    cur.close()
    return redirect('/students')
```

- `@app.route('/printcontract/<id>')` : Cette route permet d'accéder à la génération de contrat pour un étudiant identifié par son id.
- `cur.execute("SELECT * FROM students WHERE studentId=%s", (id,))` : Exécute une requête SQL pour récupérer les informations de l'étudiant correspondant à l'identifiant `id` fourni.
- `cur.execute("SELECT * FROM school")` : Exécute une requête SQL pour récupérer les informations de l'école.
- `cur.execute("SELECT * FROM companies WHERE email=%s", (student[0]['company_email'],))` : Récupère les informations de l'entreprise à partir de l'email de l'entreprise de l'étudiant.
- `datas = {...}` : Prépare un dictionnaire contenant les données de l'étudiant, de l'école et de l'entreprise, qui seront utilisées pour remplir le contrat.
- `contract = student[0]['type_contrat']` : Récupère le type de contrat de l'étudiant.
- `python`
- `fill_data_pdf(contract)` : Fonction qui génère les champs nécessaires pour remplir le contrat en fonction du type de contrat.
- `recup_data(datas)` : Fonction qui récupère les données à insérer dans les champs du PDF.
- `update_data_pdf(list_champs, doc, update_datas, contract)` : Remplit le PDF avec les données fournies.
- `cur.close()` : Ferme le curseur après avoir exécuté les requêtes SQL.
- `redirect('/students')` : Redirige vers la page des étudiants une fois le PDF généré.

e. Sécurité et bonnes pratiques

1. **Bcrypt pour le hachage des mots de passe** : une étape cruciale pour garantir la sécurité des utilisateurs, où les mots de passe ne sont jamais stockés en clair.

2. **Utilisation des variables d'environnement** : cela permet de ne jamais exposer les identifiants sensibles directement dans le code source.
3. **Validation des entrées utilisateur** : toutes les données saisies par les utilisateurs sont traitées de manière sécurisée pour éviter les attaques de type injection SQL ou XSS (Cross-Site Scripting).
4. **Documentation et tests** : chaque route et chaque fonctionnalité a été testée manuellement dans l'environnement de développement. De plus, des commentaires détaillés ont été ajoutés dans le code pour faciliter la maintenance future.

Exemple de test unitaire automatisé

Ici, un exemple de test unitaire, en utilisant le module de test intégré à Python unittest.

Objectif : Vérifier que l'utilisateur peut s'inscrire avec succès en fournissant des informations valides, et que son mot de passe est haché et stocké correctement dans la base de données.

```
import unittest
from app import app, mysql

class RegisterTest(unittest.TestCase):

    # Set up test client
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True

    def test_register(self):
        # Send POST request to the /register route with test data
        response = self.app.post('/register', data=dict(
            nom='TestUser',
            prenom='Test',
            email='testuser@example.com',
            mot_de_passe='password123',
            role='student'
        ), follow_redirects=True)

        # Check if the user is redirected to the homepage after registration
        self.assertEqual(response.status_code, 200)

        # Verify that the user has been added to the database
        cur = mysql.connection.cursor()
        cur.execute("SELECT * FROM utilisateurs WHERE email = %s", ('testuser@example.com',))
        user = cur.fetchone()
        cur.close()

        # Check if user is not None and password is hashed
        self.assertIsNotNone(user)
        self.assertNotEqual(user['mot_de_passe'], 'password123')

if __name__ == '__main__':
    unittest.main()
```

Explications du test :

1. Étapes manuelles : L'utilisateur remplit un formulaire, soumet les informations et vérifie la redirection vers la page d'accueil.
2. Vérification en base de données : Après l'inscription, une vérification est effectuée pour s'assurer que l'utilisateur est ajouté à la base de données, et que son mot de passe est haché.
3. Suivi de la fonctionnalité : Le test permet de confirmer que le processus d'inscription est fonctionnel.

Cet exemple montre à la fois comment un test manuel est réalisé et comment il pourrait être automatisé

5.3 Fonctionnalité : Remplissage de pdf

Nous avons des fonctionnalités sur notre application, notamment le remplissage automatique de contrats, ce qui facilite la gestion des documents. L'une des principales tâches que nous avons implémentées est la conversion de fichiers PDF en images. Par exemple, nous utilisons la fonction suivante pour convertir un fichier PDF en images et les encoder en base64 :


```
def convert_pdf_to_images(pdfFile):
    pdf_path = os.path.join(contrats_signes, f'{pdfFile}.pdf')
    pdf_document = fitz.open(pdf_path)
    images = []
    for page_number in range(pdf_document.page_count):
        page = pdf_document[page_number]
        image_bytes = page.get_pixmap().tobytes("png")
        image_base64 = base64.b64encode(image_bytes).decode('utf-8')
        images.append(image_base64)
    return images
```

De plus, nous avons mis en place un système de récupération des données depuis un format JSON, permettant de remplir les champs d'un PDF avec des informations spécifiques à l'entreprise, comme le nom ou le SIRET. Voici un extrait de la fonction de récupération des données :

```
def recup_data(datas):
    denomination = datas['company']['nom_entreprise']
    siret = datas['company']['siret']
    employeur_telephone = datas['company']['tel']
    # D'autres données sont également récupérées ici
    update_data = {
        8: {'Value': denomination, 'Position': 'bottom'},
        9: {'Value': siret, 'Position': 'bottom'},
        20: {'Value': employeur_telephone},
        # D'autres champs peuvent être mis à jour
    }
    return update_data
```

Ces data sont directement récupérés grâce à du scraping avec BeautifulSoup que nous verrons dans la partie 4.4

Enfin, pour assurer que les données soient correctement insérées dans le PDF, nous avons une fonction qui met à jour les champs du PDF en fonction des données récupérées. Nous pouvons ainsi gérer divers types de champs, comme les cases à cocher ou les zones de texte :

```
def update_data_pdf(unique_fields_sorted, doc, update_data, cerfa):
    for key, value in update_data.items():
        unique_fields_sorted[key].update(value)

    for i, field in enumerate(unique_fields_sorted):
        if field['Type'] == 'CheckBox' and field['Value'] == True:
            draw_check(page, field, 1)
        elif field['Type'] == 'EditBox':
            if field['Value'] != "":
                num_str = field['Value']
                doc_copy[field['PageIndex'] - 1].insert_text((field['Left'], field['Top']), num_str)
    doc_copy.save(os.path.join(contrats_signes, f'{cerfa}_filled.pdf'))
```

5.4 Fonctionnalité : Scraping avec beautifulsoup

Importation des bibliothèques

```
from bs4 import BeautifulSoup
import requests
```

Au début du code, les bibliothèques nécessaires sont importées. BeautifulSoup est utilisé pour analyser les documents HTML, et requests est utilisé pour envoyer des requêtes HTTP.

Initialisation des dictionnaires

```
resume = {}
infos = {}
activite = {}
status = {}
infos_etablissement = {
    'resume': resume,
    'informations': infos,
    'activite': activite,
    'statut': status
}
```

Plusieurs dictionnaires sont initialisés pour stocker les informations extraites, y compris les détails de l'établissement et son statut.

Définition de l'en-tête de la requête

Les en-têtes de la requête HTTP sont définis pour simuler un navigateur Web, ce qui permet de contourner certaines restrictions du serveur.

Fonction d'extraction des informations d'adresse

```
def extraire_informations_adresse(adresse):
    # Split the address into words
    mots = adresse.split()

    # Initialize variables
    numero_rue = None
    type_rue = None
    complement = None
    code_postal = None
    ville = None

    # Iterate through the words of the address
    for i, mot in enumerate(mots):
        if mot.isdigit() and i > 0:
            # If the word is a number and not the first, it's probably the street number
            numero_rue = mot
            type_rue = " ".join(mots[:i]) # Get the street type
            complement = " ".join(mots[i+1:]) # Get any additional address details
            break

    # Find the postal code and city
    for i in range(len(mots) - 1, 0, -1):
        if mots[i].isdigit():
            # If the word is a number, it is likely the postal code
            code_postal = mots[i]
            ville = " ".join(mots[i+1:]) # Join the remaining words as the city name
            break

    return numero_rue, type_rue, complement, code_postal, ville
```

La fonction `extraire_informations_adresse` prend une adresse sous forme de chaîne et la divise en différents composants : numéro de rue, type de rue, complément, code postal et ville.

Fonction de recherche d'entreprise

La fonction `rechercheEntreprise` effectue une recherche d'entreprise en utilisant un numéro SIREN. Elle construit une URL et récupère le contenu de la page, puis utilise BeautifulSoup

pour analyser les résultats.

```
def rechercheEntreprise(siren:str):  
    url = f"https://www.societe.com/cgi-bin/search?champs={siren}"  
    html_page = requests.get(url, headers=headers)  
  
    if html_page.status_code == 200:  
        soup = BeautifulSoup(html_page.content, "html.parser")  
        ;  
        resultats = soup.find_all('a', class_='ResultBloc__link__content')
```

Test de la fonction d'extraction d'adresse

Un test unitaire est inclus pour vérifier le bon fonctionnement de la fonction `extraire_informations_adresse`. Cela permet de s'assurer que la fonction renvoie les résultats attendus.

```
import unittest  
  
class TestExtractionAdresse(unittest.TestCase):  
  
    def test_extraire_informations_adresse(self):  
        adresse = "123 Rue de la République 75001 Paris"  
        numero_rue, type_rue, complement, code_postal, ville = extraire_informations_adresse(adresse)  
  
        self.assertEqual(numero_rue, "123")  
        self.assertEqual(type_rue, "Rue de la République")  
        self.assertEqual(complement, "")  
        self.assertEqual(code_postal, "75001")  
        self.assertEqual(ville, "Paris")  
  
if __name__ == "__main__":  
    unittest.main()
```

6. Déploiement de l'application

En complément du déploiement sur Google Cloud Platform (GCP), nous avons soigneusement planifié les étapes pour le déploiement de notre site web. Ce processus inclut

plusieurs phases clés, garantissant une transition fluide et efficace.

RÉSUMÉ	MISE EN RÉSEAU	SÉCURITÉ	TESTS DE CONNECTIVITÉ
Mise en réseau			
Nom de la connexion	cerfa-projet-nao:europa-west9:mysqlcerfa 		
Connectivité IP privée ?	34.195.165.111		
Connectivité IP publique ?	Désactivée		
Adresse IP publique	Désactivée		
Sécurité			
Réseaux autorisés ?	1 network		
Make IP, DNS, DNS			

6.1 Achat du nom de domaine

Pour déployer l'application en ligne, la première étape consiste à acheter un nom de domaine. Je prévois de choisir un fournisseur de nom de domaine tel que Google Domains pour acquérir un nom de domaine adapté à l'application. Le choix du nom de domaine est crucial pour garantir une bonne accessibilité et une image professionnelle du service.

6.2 Mise en place du protocole HTTPS

Une fois le nom de domaine acquis, la prochaine étape consiste à configurer le certificat SSL/TLS, qui permet de sécuriser les échanges entre l'utilisateur et le serveur via le protocole HTTPS. L'utilisation de services gratuits comme **Let's Encrypt** est envisagée pour obtenir un certificat SSL sans frais supplémentaires. L'activation du HTTPS est primordiale pour la sécurité des données, en particulier pour les informations sensibles traitées par l'application.

6.3 Hébergement

Le site sera hébergé sur le serveur cloud GCP. Il faudra relier le nom de domaine à l'adresse IP du serveur, permettant aux utilisateurs d'accéder à l'application via une URL simple

6.4 Déploiement des sources et configuration finale

Après l'achat du nom de domaine et la mise en place des certificats SSL, je procéderai au déploiement de l'application. Cette étape inclut l'installation des dépendances Python (Flask, MySQL), la configuration de la base de données, et le déploiement des sources de l'application sur le serveur.

7. Après projet

Pour rester informé des évolutions des outils que j'utilise dans le développement et des questions de sécurité, je consulte régulièrement plusieurs sources fiables et variées.

7.1 ANSSI (Agence nationale de la sécurité des systèmes d'information) : Je m'appuie sur l'ANSSI pour rester informé des dernières pratiques en matière de sécurité des systèmes d'information. Cet organisme offre des ressources complètes, comme des guides de bonnes

pratiques et des recommandations pour la protection des systèmes d'information. Je consulte régulièrement leurs documents et leurs outils pour identifier les failles potentielles dans mes projets et appliquer des correctifs.

7.2 Stack Overflow : Lorsque je rencontre un problème technique, je me tourne souvent vers Stack Overflow, une plateforme où je peux trouver des solutions à des problèmes rencontrés par d'autres développeurs. J'y trouve des réponses précises sur les bugs, les bonnes pratiques de programmation et les nouveautés des outils de développement. Cette communauté active m'aide à rester à jour sur les frameworks et les langages que j'utilise.

7.3 GitHub : J'utilise GitHub non seulement pour héberger et collaborer sur des projets, mais aussi pour suivre les évolutions des bibliothèques et des frameworks que j'intègre dans mes applications. Je surveille les dépôts de projets open source, notamment ceux des outils que j'utilise, pour être informé des nouvelles versions, des patchs de sécurité et des discussions autour des vulnérabilités.

En me basant sur ces trois ressources, je m'assure de rester constamment informé des nouveautés et des meilleures pratiques dans le domaine du développement et de la sécurité informatique.

8. Application desktop : Gestionnaire de mots de passe

8.1 Description de l'application :

Afin de simplifier la gestion des mots de passe au sein de l'entreprise, nous avons développé une application de bureau intuitive en Python en utilisant la librairie Tkinter. Cette application permet d'enregistrer les différents mots de passe utilisés pour divers sites ou applications, renforçant ainsi la sécurité des comptes.

Grâce à notre solution, les utilisateurs bénéficient d'une interface conviviale sur leur propre ordinateur, simplifiant la gestion des accès et éliminant la nécessité de recourir à des méthodes moins sécurisées telles que les notes post-it. L'application assure un stockage sécurisé des mots de passe en utilisant des techniques de hachage, garantissant la confidentialité des informations sensibles.

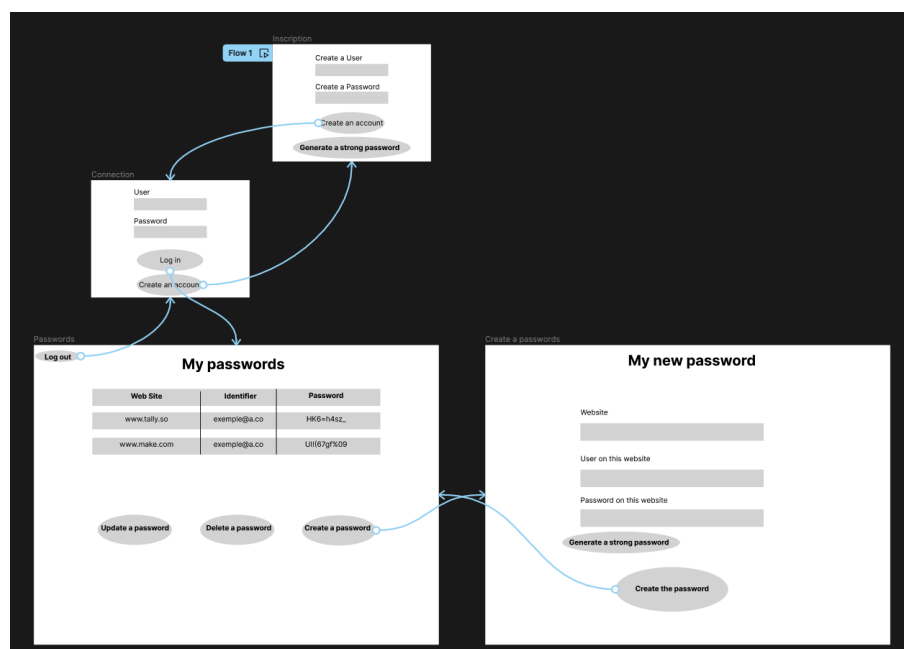


Schéma de l'application Desktop

Voici le code qui nous permet de hasher les mots de passe

```
# Function to encrypt a password
def encrypt_password_for_user(password):
    # Generate a random salt
    salt = bcrypt.gensalt()

    # Encrypt the password with the salt
    hashed_password = bcrypt.hashpw(password.encode(), salt)

    return hashed_password

def check_password(input_password, hashed_password):
    if (not input_password or hashed_password == None): return False
    # Check if plaintext password matches the hashed password
    return bcrypt.checkpw(input_password.encode(), hashed_password)
```

L'une des fonctionnalités clés de notre application de bureau est la génération automatique de mots de passe forts avec le bouton “generate a strong password”, éliminant ainsi les risques liés à l'utilisation de mots de passe faibles ou réutilisés. Cette approche renforce la robustesse de la sécurité tout en offrant une expérience utilisateur fluide. Ci-dessous le code qui nous permet de générer des mots de passe puissants.

```
def generate_strong_password(length=12, password_e=None):
    # Check if password_e is specified
    if password_e is None:
        raise ValueError("Password entry is not specified !")

    # Characters to use for generating the password
    chars = string.ascii_letters + string.digits + string.punctuation

    # Ensure at least one character from each category is included in the password
    password = ''.join(random.choice(string.ascii_lowercase) for _ in range(3))
    password += ''.join(random.choice(string.ascii_uppercase) for _ in range(3))
    password += ''.join(random.choice(string.digits) for _ in range(2))
    password += ''.join(random.choice(string.punctuation) for _ in range(2))

    # Add random characters until desired length
    password += ''.join(random.choice(chars) for _ in range(length - len(password)))

    # Shuffle characters to increase entropy
    password_list = list(password)
    random.shuffle(password_list)
    password = ''.join(password_list)

    # Clear existing content of password entry and insert the newly generated password
    if password_e:
        password_e.delete(0, tk.END)
        password_e.insert(0, password)

    return password
```

En centralisant tous les mots de passe au même endroit, notre application facilite l'accès aux différents comptes, contribuant ainsi à l'efficacité opérationnelle. De plus, elle réduit le temps consacré à la récupération des mots de passe oubliés, améliorant la productivité des utilisateurs.

En somme, notre application de bureau offre une solution complète pour une gestion efficace et sécurisée des mots de passe, contribuant à renforcer la sécurité des comptes au sein de l'entreprise et simplifiant les opérations quotidiennes d'authentification en ligne. Elle vient parfaitement compléter notre application web en facilitant la gestion des mots de passe des différents outils (Tally, Make, et notre application), ajoutant ainsi une couche supplémentaire de sécurité.

8.2 Base de données de l'application :

```
8  # Function to initialize the database
9  def init_database():
10     try:
11         conn = sqlite3.connect("PM.db")
12         cursor = conn.cursor()
13         # Creating passwords table if not exists
14         cursor.execute("""
15         CREATE TABLE IF NOT EXISTS passwords (
16             id INTEGER PRIMARY KEY,
17             site TEXT NOT NULL UNIQUE,
18             username TEXT NOT NULL,
19             password TEXT NOT NULL
20         )
21         """)
22         # Creating users table if not exists
23         cursor.execute('''CREATE TABLE IF NOT EXISTS users
24             (id INTEGER PRIMARY KEY,
25              username TEXT NOT NULL UNIQUE,
26              password TEXT NOT NULL)''')
27
28         # Creating configs table if not exists
29         cursor.execute('''CREATE TABLE IF NOT EXISTS configs
30             (id INTEGER PRIMARY KEY,
31              key TEXT NOT NULL UNIQUE,
32              value TEXT NOT NULL)''')
33         conn.commit()
34         u.init_key()
35     except sqlite3.Error as e:
36         tk.messagebox.showerror("Error", f"Database error: {e}")
37     finally:
38         if conn:
39             conn.close()
```

La fonction `init_database()` :

- Établit une connexion à la base de données SQLite nommée "PM.db".
- Crée une table appelée "passwords" pour stocker les mots de passe des utilisateurs, avec les colonnes id, site, username et password.
- Crée une table appelée "users" pour stocker les informations des utilisateurs, avec les colonnes id, username et password.
- Crée une table appelée "configs" pour stocker des configurations diverses, avec les colonnes id, key et value.
- Commence une transaction pour exécuter toutes ces instructions SQL, et la valide en utilisant `conn.commit()`.
- Appelle la fonction `u.init_key()` (qui n'est pas définie ici) pour initialiser une clé.
- Gère les erreurs éventuelles en affichant un message d'erreur via Tkinter s'il y a un problème lors de l'exécution des instructions SQL.
- Enfin, ferme la connexion à la base de données si elle est ouverte.

Cette fonction est donc responsable de créer les tables nécessaires dans la base de données lorsqu'elle est appelée, garantissant ainsi que la structure de la base de données

est correctement mise en place pour que l'application puisse stocker et récupérer les données correctement.

8.3 Fonction pour s'enregistrer (Log in)

```
def login_interface():
    login_window = tk.Tk()
    login_window.title("Connection Interface")
    login_window.geometry('350x200')

    # Locking the dimensions of the main window
    login_window.resizable(False, False)

    login_frame = tk.Frame(login_window)
    login_frame.pack(padx=20, pady=20)

    ttk.Label(login_frame, text="Login", font=("Helvetica", 16)).grid(row=0, columnspan=3)

    ttk.Label(login_frame, text="Username:").grid(row=1, column=0)
    username_entry = ttk.Entry(login_frame)
    username_entry.grid(row=1, column=1, pady=10)

    ttk.Label(login_frame, text="Password:").grid(row=2, column=0)
    password_entry = ttk.Entry(login_frame, show="*")
    password_entry.grid(row=2, column=1)

    ttk.Button(login_frame, text="Login", command=lambda: login(username_entry.get(), password_entry.get(), login_window)).grid(row=3, column=0, columnspan=2, pady=10, sticky="w")
    ttk.Button(login_frame, text="Register", command=lambda: register_interface()).grid(row=3, column=1, columnspan=2, pady=10, sticky="e")

    login_window.mainloop()
```

Cette fonction crée une interface de connexion pour l'application en utilisant le module Tkinter de Python. Voici une explication concise :

- La fonction login_interface() :
 - Crée une fenêtre principale avec le titre "Interface de connexion" et une taille de 350x200 pixels.
 - Verrouille les dimensions de la fenêtre principale pour qu'elle ne puisse pas être redimensionnée.
 - Crée un cadre (frame) pour organiser les widgets de manière ordonnée à l'intérieur de la fenêtre.
 - Ajoute des étiquettes (labels) pour indiquer les champs "Nom d'utilisateur" et "Mot de passe", ainsi que des zones de saisie (entry) correspondantes pour que l'utilisateur puisse entrer ses informations de connexion.
 - Utilise des boutons pour permettre à l'utilisateur de se connecter ou de s'inscrire. Ces boutons appellent respectivement les fonctions login() et register_interface() (qui ne sont pas définies ici).
 - La fonction login() est appelée avec les informations de connexion entrées par l'utilisateur.
 - La fonction register_interface() est appelée pour afficher l'interface d'inscription.
 - Enfin, la fonction mainloop() est appelée pour démarrer la boucle d'événements Tkinter, ce qui permet à l'interface utilisateur de répondre aux interactions de l'utilisateur jusqu'à ce que la fenêtre soit fermée.

8.4 Changer un mot de passe : Update a password

```
def update_password_interface(id,site_e,username_e,password_e,tree_e):
    # Creating "update password" window
    update_password_window = tk.Toplevel()
    update_password_window.title("Edit password")
    update_password_window.geometry("350x200")

    # Locking the dimensions of the main window
    update_password_window.resizable(False, False)

    # Creating style
    style = ttk.Style()
    style.configure("TButton", foreground="black", background="lightgrey")

    # Creating main frame
    update_password_frame = ttk.Frame(update_password_window)
    update_password_frame.pack(padx=20, pady=20, fill=tk.BOTH, expand=True)

    # Creating labels and entries
    ttk.Label(update_password_frame, text="Site:").grid(row=0, column=0, pady=10, sticky="e")
    site_entry = ttk.Entry(update_password_frame)
    site_entry.grid(row=0, column=1, padx=10, sticky="w")
    site_entry.insert(tk.END, site_e)

    ttk.Label(update_password_frame, text="Username:").grid(row=1, column=0, pady=10, sticky="e")
    username_entry = ttk.Entry(update_password_frame)
    username_entry.grid(row=1, column=1, padx=10, sticky="w")
    username_entry.insert(tk.END, username_e)

    ttk.Label(update_password_frame, text="Password:").grid(row=2, column=0, pady=10, sticky="e")
    password_entry = ttk.Entry(update_password_frame, show="*")
    password_entry.grid(row=2, column=1, padx=10, sticky="w")
    password_entry.insert(tk.END, password_e)

    # Creating buttons
    generate_button = ttk.Button(update_password_frame, text="Generate",
                                command=lambda: u.generate_strong_password(12, password_entry))
    generate_button.grid(row=3, column=0, pady=10, padx=10, sticky="w")
```

Cette fonction crée une interface pour permettre à l'utilisateur de mettre à jour un mot de passe existant dans une application. Voici une explication concise :

- La fonction `update\_password\_interface(id, site\_e, username\_e, password\_e, tree\_e)` :
 - Crée une fenêtre modale (une sous-fenêtre) avec le titre "Modifier le mot de passe" et une taille de 350x200 pixels.
 - Verrouille les dimensions de cette fenêtre pour qu'elle ne puisse pas être redimensionnée.
 - Crée un cadre (frame) pour organiser les widgets de manière ordonnée à l'intérieur de la fenêtre.
 - Configure un style pour les boutons.
 - Ajoute des étiquettes (labels) pour indiquer les champs "Site", "Nom d'utilisateur" et "Mot de passe", ainsi que des zones de saisie (entry) correspondantes pour afficher les informations du mot de passe à modifier.

- Les informations de site, de nom d'utilisateur et de mot de passe passées en paramètres ('site_e', 'username_e', 'password_e') sont insérées dans les zones de saisie correspondantes.
- Crée un bouton "Générer" qui appelle une fonction pour générer un mot de passe fort et le place dans la zone de saisie du mot de passe.
- Crée un bouton "Mettre à jour" qui appelle une fonction pour mettre à jour le mot de passe dans la base de données, puis ferme la fenêtre modale.
- Verrouille l'accès à la fenêtre principale jusqu'à ce que la fenêtre modale soit fermée, pour empêcher l'interaction avec la fenêtre principale tant que l'édition du mot de passe n'est pas terminée.
- Attends que la fenêtre modale soit fermée avant de continuer l'exécution du programme.

En résumé, cette fonction fournit une interface conviviale pour que l'utilisateur puisse visualiser et mettre à jour les informations d'un mot de passe stocké dans l'application.

8.5 Technos utilisées

Pour réaliser cette application desktop, les technologies et outils suivants ont été utilisés :

- **Langage de programmation** : Python
- **Bibliothèque GUI** : Tkinter (pour l'interface graphique)
- **Base de données** : SQLite (pour la gestion des données)
- **Gestion des versions** : GitHub (pour le contrôle de version et la gestion du code)
- **Éditeur de code** : Visual Studio Code
- **Modules supplémentaires** :
 - pyperclip (pour la gestion du presse-papiers)
 - ttk (pour les widgets avancés de Tkinter)

Ces outils et bibliothèques permettent de créer une application de gestion de mots de passe avec une interface utilisateur conviviale et une base de données sécurisée.

ANNEXES

Application mobile

Mon application mobile est une application React Native utilisant Expo. Voici une description étape par étape de sa configuration :

1. Informations générales sur mon application

- **Nom de l'application** : "FAQ"
C'est le nom que j'ai donné à mon application, il sera affiché sur l'appareil.
- **Slug** : "HelloWorld"
C'est un identifiant unique utilisé par Expo pour gérer mon application en ligne.
- **Version** : "1.0.0"
J'ai défini la première version de mon application. Elle suit la convention SemVer.

```
{
  "expo": {
    "name": "HelloWorld",
    "slug": "HelloWorld",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "light",
    "splash": {
      "image": "./assets/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
  },
}
```

Mode portrait : J'ai configuré mon application pour qu'elle fonctionne uniquement en mode portrait.

J'ai choisi une icône pour mon application, stockée dans le dossier assets sous le nom icon.png.

J'ai décidé d'utiliser un **thème clair** pour mon application.

J'ai configuré un écran de démarrage avec une image que j'ai placée dans le dossier assets/splash.png.

L'image est redimensionnée pour s'adapter à l'écran tout en conservant ses proportions, avec un fond blanc pour garder un design propre et simple.

6. Compatibilité iOS

J'ai fait en sorte que mon application soit **compatible avec les tablettes iOS** (iPad), pour que l'expérience utilisateur soit fluide sur ces appareils.

```
    "ios": {  
      "supportsTablet": true  
    }
```

7. Compatibilité Android

J'ai configuré une icône adaptative spécifique pour Android, avec une image au premier plan et un fond blanc.

```
    "android": {  
      "adaptiveIcon": {  
        "foregroundImage": "./assets/adaptive-icon.png",  
        "backgroundColor": "#ffffff"  
      }  
    }
```

8. Compatibilité Web

J'ai aussi pris en compte le côté web en ajoutant un favicon pour les navigateurs, que j'ai placé dans assets/favicon.png.

```
  },  
  "web": {  
    "favicon": "./assets/favicon.png"  
  }  
}
```

Structure de mon projet :

- **Dossier assets** : Ce dossier contient toutes les images de l'application, comme les icônes et le splash screen.
- **Dossier node_modules** : Il contient toutes les dépendances nécessaires que j'ai installées via npm ou yarn.

- **App.js** : Le fichier principal où je code l'application en React Native.
- **package.json** : Il contient les informations sur mon projet, les dépendances que j'utilise et les scripts pour lancer l'application.

Et voici ce que contient mon application dans le dossier : App.js :

```
1 > import React, { useState } from 'react';--
3
4 > const DATA = { ...
71 };
72
73 > export default function App() { ...
143 }
144
145 > const styles = StyleSheet.create({ ...
197 });|
```

Exemples de questions / réponses

```
const DATA = {
  entreprises: [
    { id: 1, question: 'Quelles sont les démarches administratives pour embaucher un apprenti en alternance ?', answer: 'Les entreprises doivent s'adresser au service des ressources humaines de l'entreprise pour connaître les démarches à suivre.' },
    { id: 2, question: 'Quels sont les avantages fiscaux pour l'embauche d'un apprenti ?', answer: 'Les entreprises bénéficient d'exonérations de cotisations sociales et d'un crédit d'impôt de 130% sur les cotisations patronales de sécurité sociale.' },
    { id: 3, question: 'Comment rédiger et remplir correctement un contrat CERFA pour un alternant ?', answer: 'Vous pouvez trouver le modèle de contrat CERFA sur le site du Service Public ou auprès de votre organisme de formation.' },
    { id: 4, question: 'Quels sont les critères d'éligibilité pour embaucher un alternant ?', answer: 'L\'entreprise doit être soumise à la législation du travail, avoir des salariés, et proposer une formation professionnelle adaptée à l'alternant.' },
    { id: 5, question: 'Comment déterminer le salaire d'un apprenti ?', answer: 'Le salaire est déterminé en fonction du niveau de qualification, de l'ancienneté et du secteur d'activité.' },
    { id: 6, question: 'Quels sont les obligations légales de l'entreprise envers l'apprenti ?', answer: 'L\'entreprise doit assurer la formation théorique et pratique de l'apprenti, lui fournir un logement, et respecter les règles de sécurité.' },
    { id: 7, question: 'Comment trouver un organisme de formation adapté à l'alternant ?', answer: 'Vous pouvez consulter le site du Service Public ou contacter votre organisme de formation.' },
    { id: 8, question: 'Quels sont les documents nécessaires pour compléter le dossier CERFA ?', answer: 'Les documents nécessaires sont : le contrat CERFA, le certificat de formation, le certificat de stage, et le certificat de travail.' },
    { id: 9, question: 'Comment s'assurer que l'alternant suit bien sa formation théorique ?', answer: 'L\'entreprise doit organiser des évaluations régulières de l'apprenti, et lui fournir un suivi personnalisé.' },
    { id: 10, question: 'Quelle est la durée minimum et maximum d'un contrat d'alternance ?', answer: 'La durée varie en fonction du niveau de qualification, de l'ancienneté et du secteur d'activité.' },
    { id: 11, question: 'Quelles sont les sanctions en cas de non-respect du contrat d'alternance ?', answer: 'Les sanctions peuvent aller de l'avertissement à la rupture du contrat, et même à des poursuites judiciaires.' },
    { id: 12, question: 'Comment se déroule la période d'essai pour un contrat d'alternance ?', answer: 'La période d'essai est de 15 jours pour les apprentis de moins de 18 ans, et de 30 jours pour les autres.' },
    { id: 13, question: 'Que faire en cas de rupture anticipée du contrat d'alternance ?', answer: 'Il faut notifier l'organisme de formation et le service des ressources humaines de l'entreprise.' },
    { id: 14, question: 'Comment gérer les absences et les congés de l'apprenti ?', answer: 'Les absences doivent être déclarées à l'organisme de formation, et les congés doivent être pris conformément à la législation du travail.' },
    { id: 15, question: 'Quel est le rôle du maître d'apprentissage dans l'entreprise ?', answer: 'Le maître d'apprentissage est responsable de la formation pratique de l'apprenti, et de son suivi personnalisé.' },
    { id: 16, question: 'Comment former et accompagner un apprenti efficacement ?', answer: 'Il est important de proposer une formation adaptée à l'alternant, et de lui fournir un suivi personnalisé.' },
    { id: 17, question: 'Quels sont les outils pour évaluer les compétences de l'apprenti ?', answer: 'Les outils utilisés sont : les évaluations régulières, les certificats de stage, et les certificats de travail.' },
    { id: 18, question: 'Quels sont les droits et obligations de l'apprenti au sein de l'entreprise ?', answer: 'L'apprenti a droit à un salaire, à une formation, et à des congés. Il est tenu de respecter les règles de sécurité, et de fournir un travail de qualité.' },
    { id: 19, question: 'Comment renouveler ou prolonger un contrat d'alternance ?', answer: 'Il faut signer un avenant au contrat, et le faire valider par l'organisme de formation.' },
    { id: 20, question: 'Quels sont les financements disponibles pour les entreprises embauchant des alternants ?', answer: 'Les entreprises peuvent bénéficier de subventions, de crédits d'impôt, et de prêts à taux réduits.' }
  ],
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    paddingTop: 100,
    paddingBottom: 50,
    paddingHorizontal: 20,
    backgroundColor: '#f5f5f5',
  },
  buttonContainer: {
    flexDirection: 'row',
    justifyContent: 'space-around',
    marginBottom: 20,
  },
  button: {
    padding: 10,
    backgroundColor: '#007BFF',
    borderRadius: 5,
  },
  activeButton: {
    backgroundColor: '#0056b3',
  },
  buttonText: {
    color: 'white',
    fontSize: 16,
  },
});
```

Rendu de l'application mobile :

